DOCKER CHALLENGE — COMPLETE PRACTICAL DOCUMENTATION

Objective

To create Docker images, push them to Docker Hub and Amazon ECR, install Terraform and AWS CLI, provision an EC2 instance using Terraform, install Jenkins, automate Docker image building using Jenkins, and deploy applications using Docker Compose.

Requirements

- Laptop or computer running Windows
- AWS account
- Existing Docker EC2 instance
- Existing Jenkins EC2 instance
- Docker Hub account
- Git
- Terraform
- AWS CLI
- Docker Engine
- Jenkins
- Python, frontend, Java, and Node.js source code

Source repositories:

- Python: https://github.com/betawins/Python-app.git
- Application source codes: https://github.com/betawins/docker-tasks.git

Docker Hub username: "chvkarthik"

AWS region: "us-east-1"

TASK 1: CREATE A CUSTOMIZED DOCKER IMAGE USING A DOCKERFILE

Objective

To create a customized Docker image for a Python application.

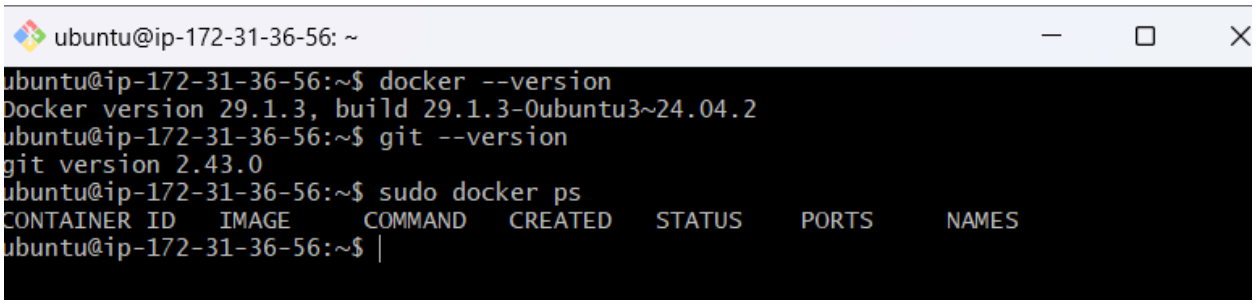
Step 1: Connect to Docker-server

1. Open AWS Console.
2. Search for EC2.
3. Click Instances.
4. Select the existing "Docker-server" instance.
5. Click Connect.
6. Select EC2 Instance Connect.
7. Click Connect.

Step 2: Verify Docker and Git

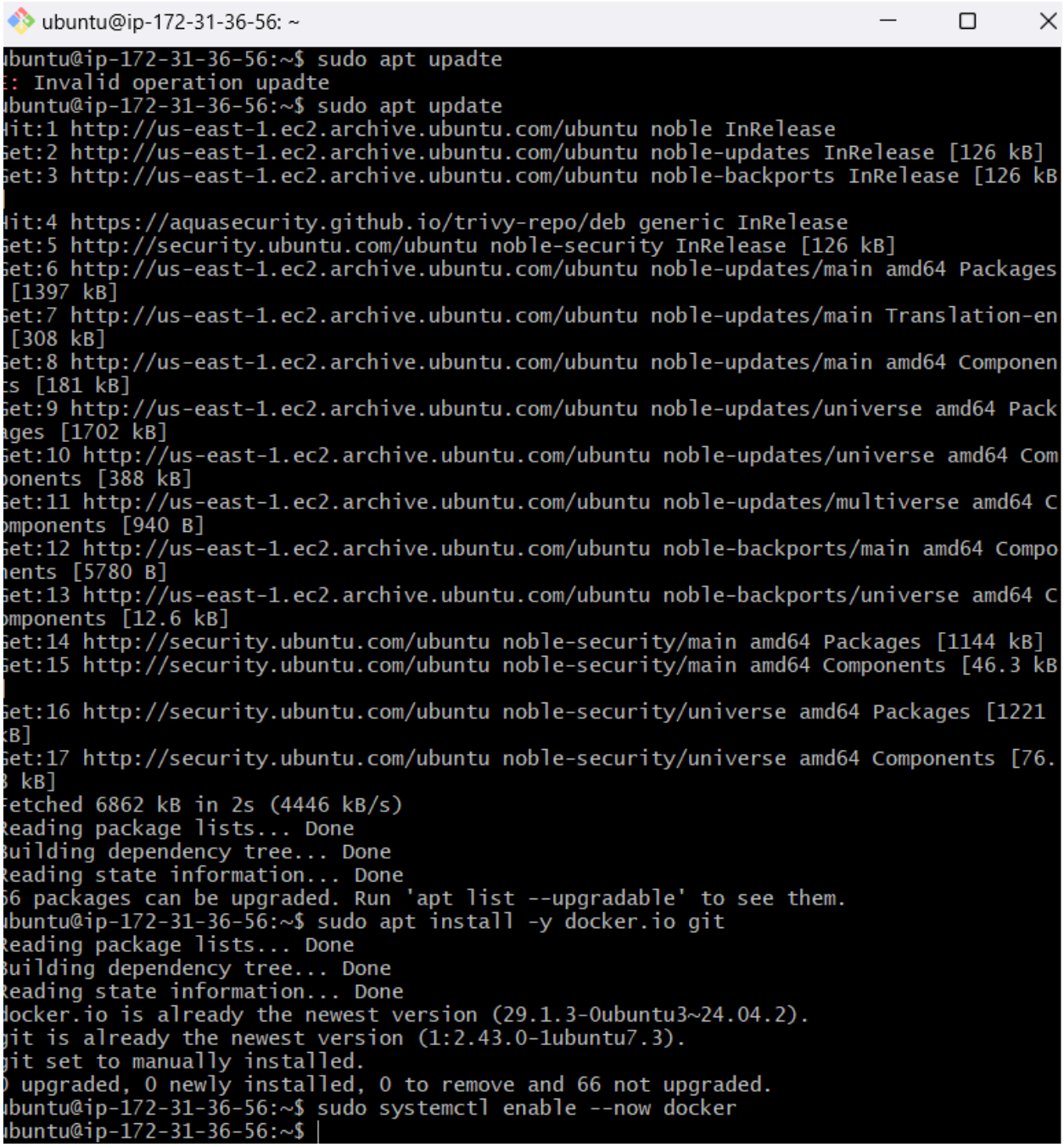
Execute:

docker --version
git --version
Sudo docker ps



If Docker is not installed on Ubuntu:

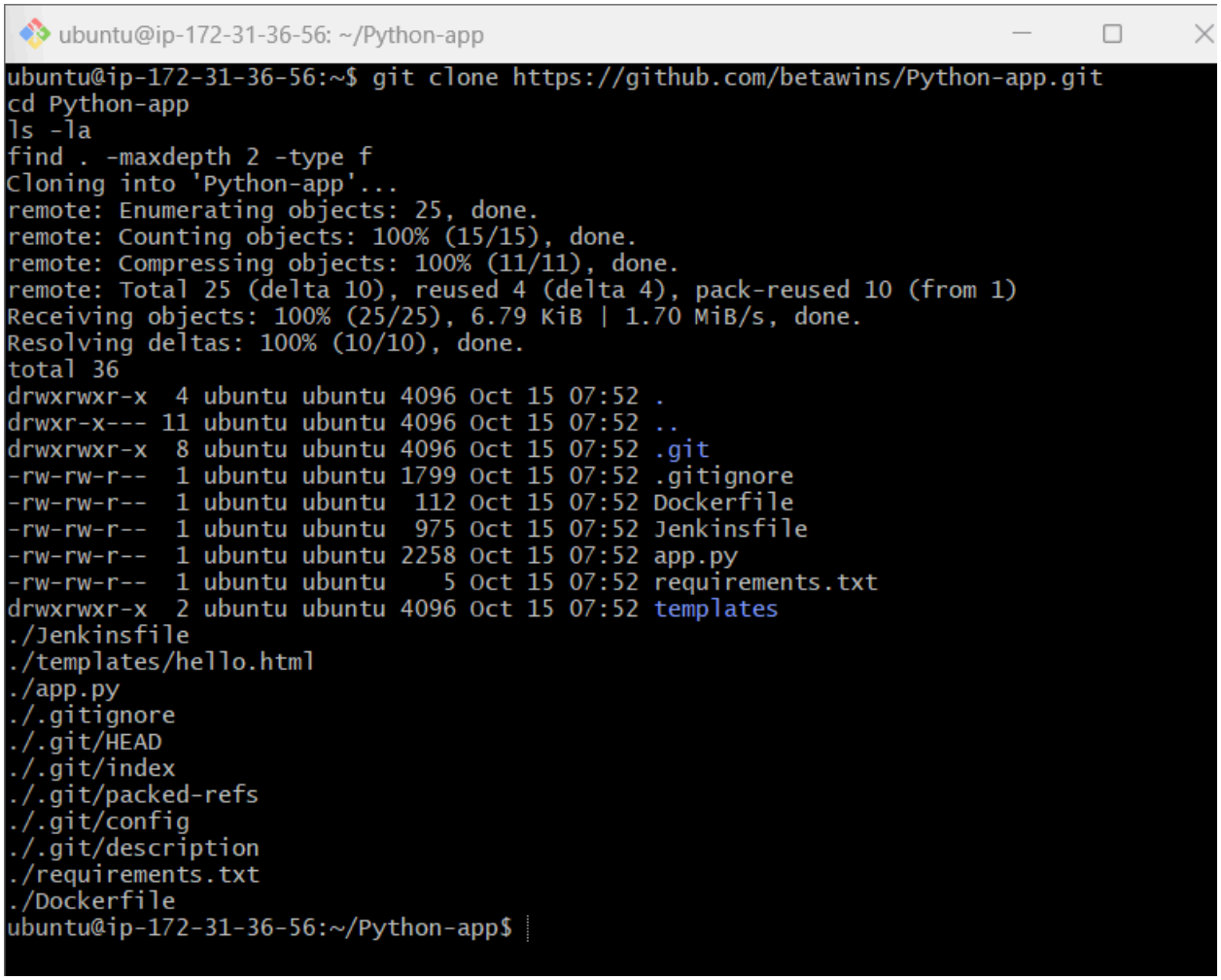
sudo apt update
sudo apt install -y docker.io git
sudo systemctl enable --now docker



Step 3: Clone the Python source code

git clone https://github.com/betawins/Python-app.git
cd Python-app
ls -la
find . -maxdepth 2 -type f

Identify the Python entry-point file, dependency file, and application port.



Step 4: Create a Dockerfile

nano Dockerfile

Example for a project containing "requirements.txt" and "app.py":

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Save using "Ctrl + O", press Enter, and exit using "Ctrl + X".

Adjust the entry-point filename and port if required by the actual source code.

Step 5: Build the image

Sudo docker build -t python-custom:v1 .
Sudo docker images

```
ubuntu@ip-172-31-36-56:~/Python-app$ sudo docker images
                                                    i Info →  U  In Use
IMAGE                                   ID             DISK USAGE   CONTENT SIZE   EXTRA
686194638896.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
                                        1586db5eaf15        503MB         135MB    U
alpine-tomcat:v1                        9b13fdf855d3        317MB        86.6MB    U
multi-stage-example:multi               1586db5eaf15        503MB         135MB    U
multi-stage-example:single              69efd96df1d5        988MB         324MB    U
mysql:8.0                               7dcddc01f13b        1.1GB         249MB    U
nginx:alpine                            df221db836e1       94.4MB        27.2MB    U
python-custom:v1                        a088934c49a0        198MB        48.3MB
python:3.12-slim                        a6e34c598f24        179MB        45.4MB
redis:alpine                            3811787313eb        161MB        39.6MB    U
ubuntu:latest                           f144425ff09b        162MB        45.6MB    U
wordpress:latest                        81a34254acaa       1.12GB         284MB    U
ubuntu@ip-172-31-36-56:~/Python-app$
```

Step 6: Run the container

docker run -d --name python-app -p 5000:8080 python-custom:v1
docker ps
docker logs python-app
curl http://localhost:5000

```
ubuntu@ip-172-31-36-56:~/Python-app$ curl http://localhost:5000
<!doctype html>
<title>Hello from Flask</title>
<body style="background: #2980b9;"></body>
<div style="color: #e4e4e4;
    text-align:  center;
    height: 90px;
    vertical-align:  middle;">

  <h1>Hello, s3cloudhub</h1>

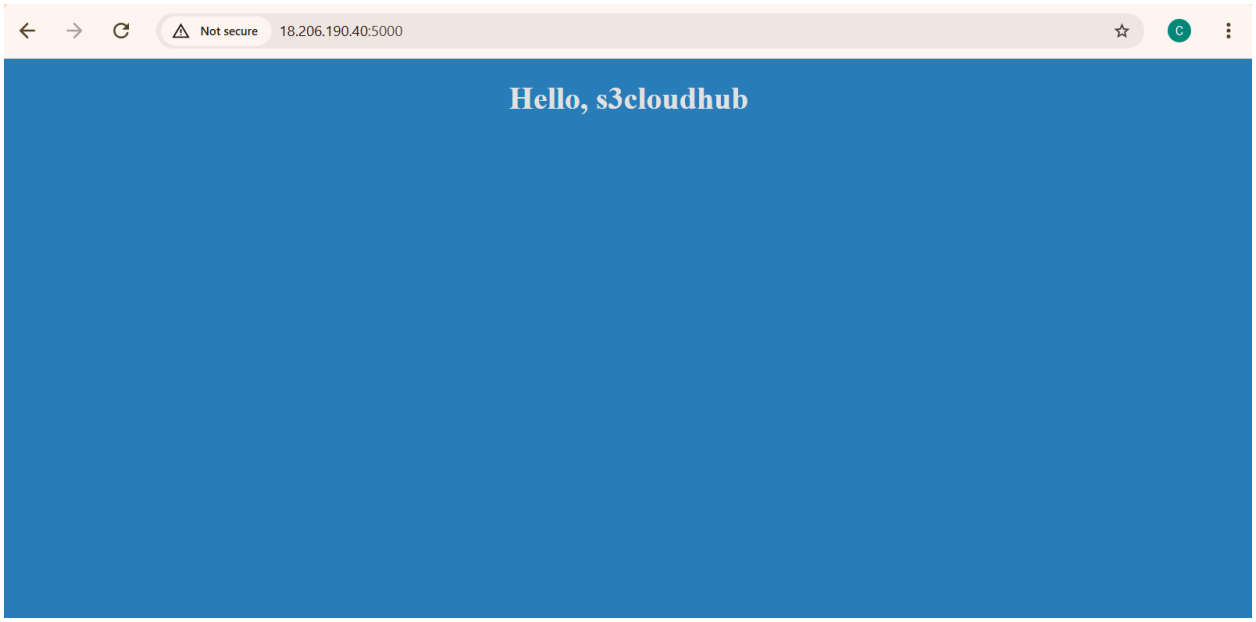



</div>ubuntu@ip-172-31-36-56:~/Python-app$
```

Result

The customized Python Docker image has been created and run.

http://PUBLIC_IP:5000



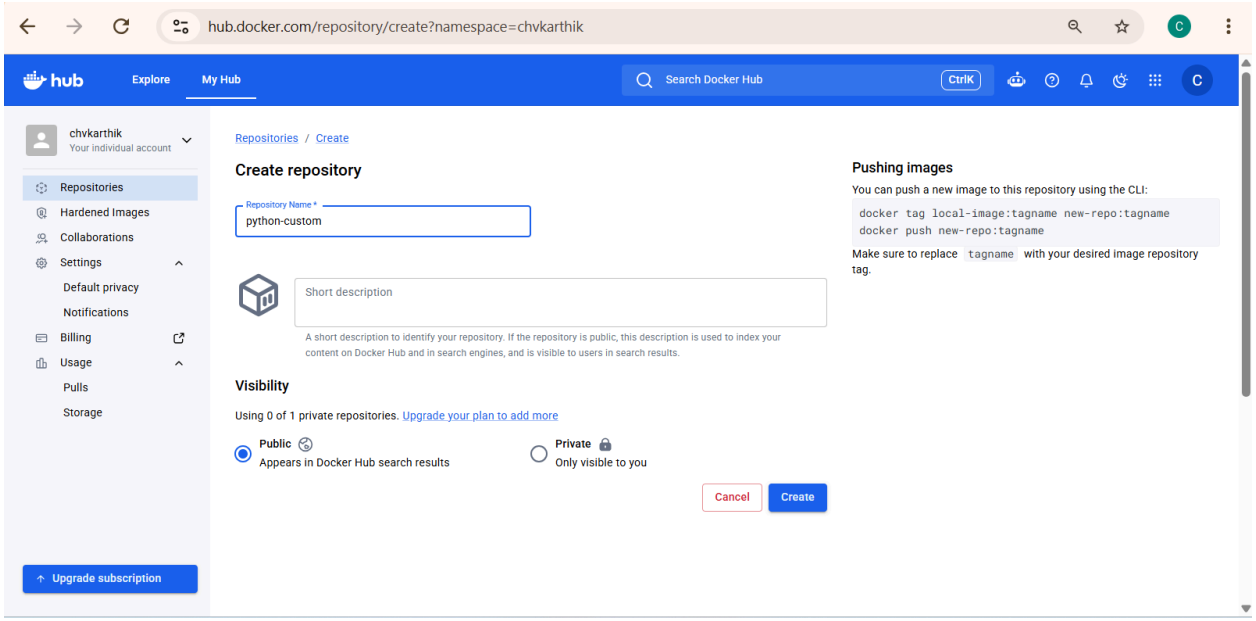
TASK 2: PUSH THE IMAGE TO DOCKER HUB

Objective

To upload the Python Docker image to Docker Hub.

Step 1: Create a repository

1. Open https://hub.docker.com/.
2. Log in.
3. Click Repositories.
4. Click Create repository.
5. Enter "python-custom".
6. Click Create.



Step 2: Log in

On Docker-server, execute:

docker login -u chvkarthik

Enter your Docker Hub access token when prompted.

```
ubuntu@ip-172-31-36-56:~$ docker login -u chvkarthik

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:

WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json
.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-172-31-36-56:~$
```

Step 3: Tag the image

Sudo docker tag python-custom:v1 chvkarthik/python-custom:v1

Step 4: Push the image

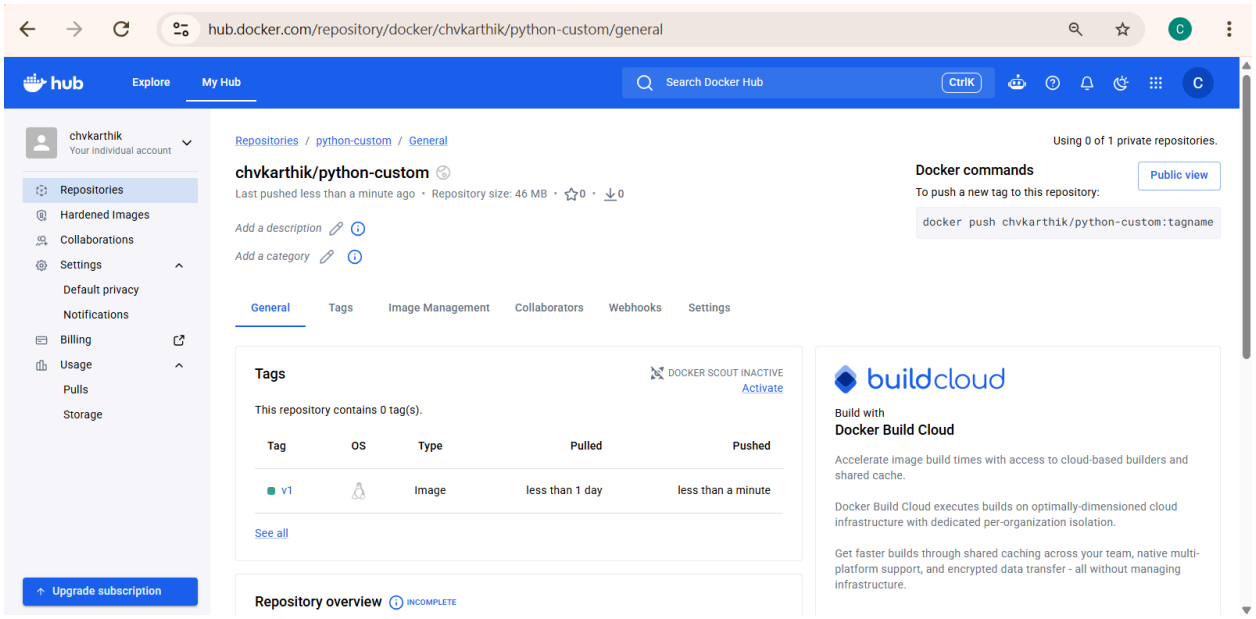
Sudo docker push chvkarthik/python-custom:v1

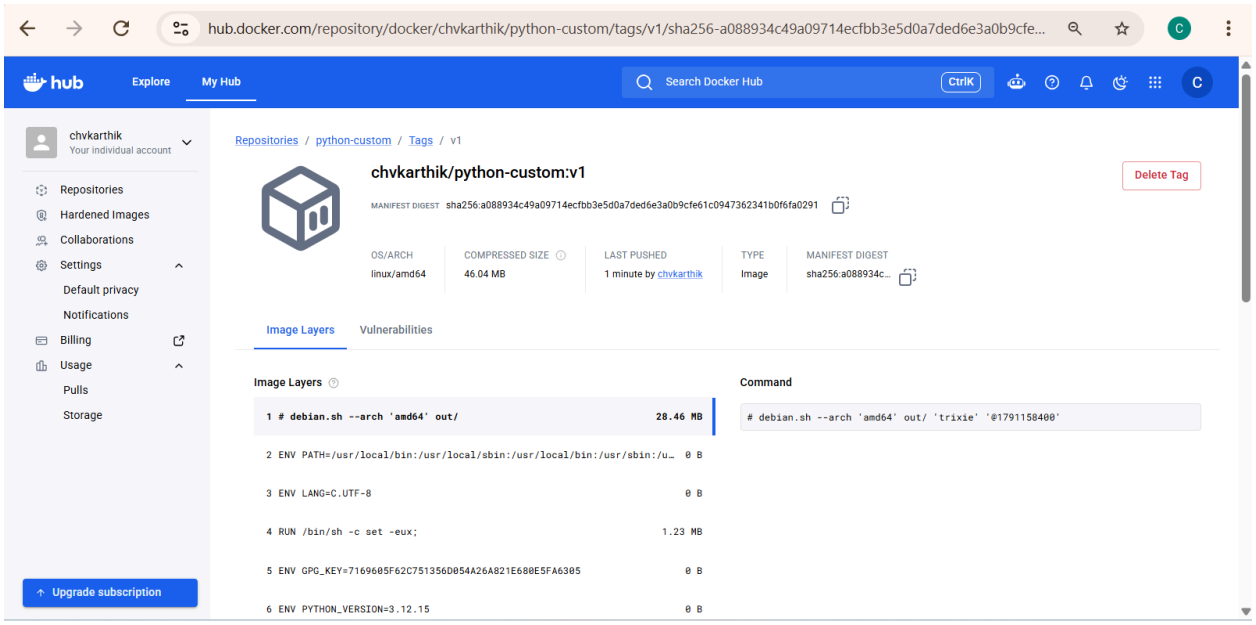
Step 5: Verify

Open Docker Hub → Repositories → "python-custom" → Tags. Verify the "v1" tag.

Result

The Python image has been pushed to Docker Hub.





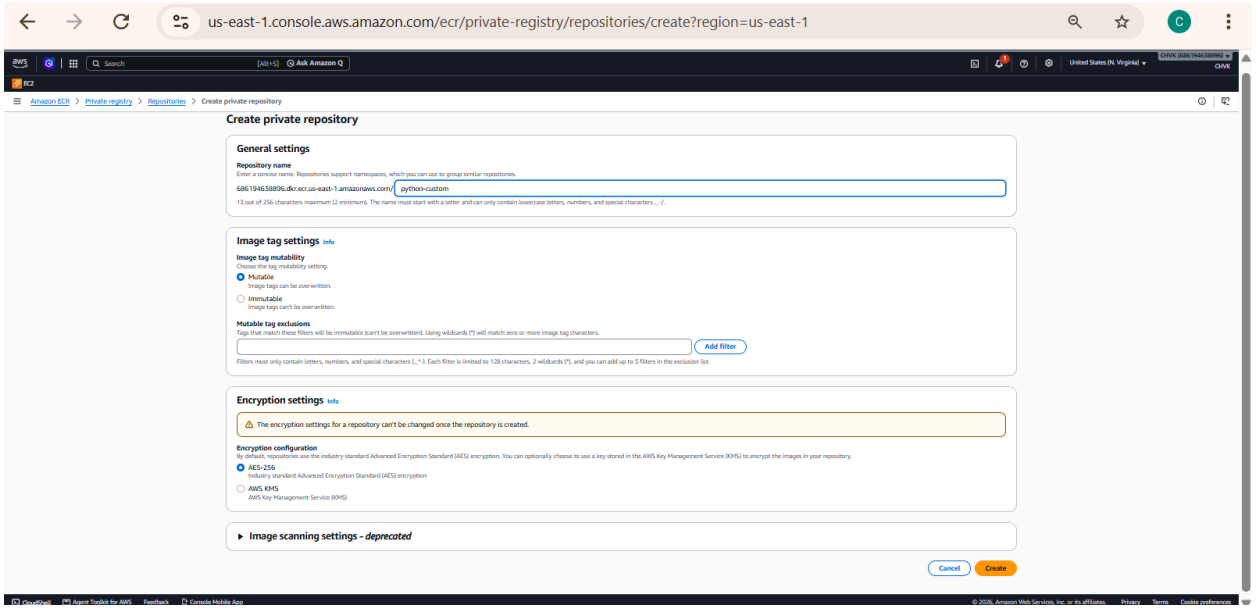
TASK 3: PUSH THE SAME IMAGE TO AMAZON ECR

Objective

To upload the same Python image to Amazon Elastic Container Registry.

Step 1: Create an ECR repository

1. Open AWS Console.
2. Search for Amazon ECR.
3. Click Repositories.
4. Click Create repository.
5. Select Private.
6. Enter "python-custom".
7. Select region "us-east-1".
8. Click Create repository.



Step 2: Verify AWS access on Docker-server

aws sts get-caller-identity

The EC2 instance must have an IAM role with the required ECR permissions. If AWS CLI is not installed on Docker-server, install it there or use another machine with properly configured AWS access.

```
ubuntu@ip-172-31-36-56:~$ aws sts get-caller-identity
{
    "UserId": "AROAZ7RDDIAYG6TFMEJ3V:i-0173134df62f99465",
    "Account": "686194638896",
    "Arn": "arn:aws:sts::686194638896:assumed-role/EC2-ECR-push-role/i-0173134df62f994
65"
}
ubuntu@ip-172-31-36-56:~$
```

Step 3: Log in to ECR

Example account ID: "686194638896".

aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 686194638896.dkr.ecr.us-east-1.amazonaws.com

Replace the account ID if necessary.

```
ubuntu@ip-172-31-36-56:~$ aws ecr get-login-password --region us-east-1 | docker login
 --username AWS --password-stdin 686194638896.dkr.ecr.us-east-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'
.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-172-31-36-56:~$
```

Step 4: Tag the image

Sudo docker tag python-custom:v1 686194638896.dkr.ecr.us-east-1.amazonaws.com/python-custom:v1

Step 5: Push the image

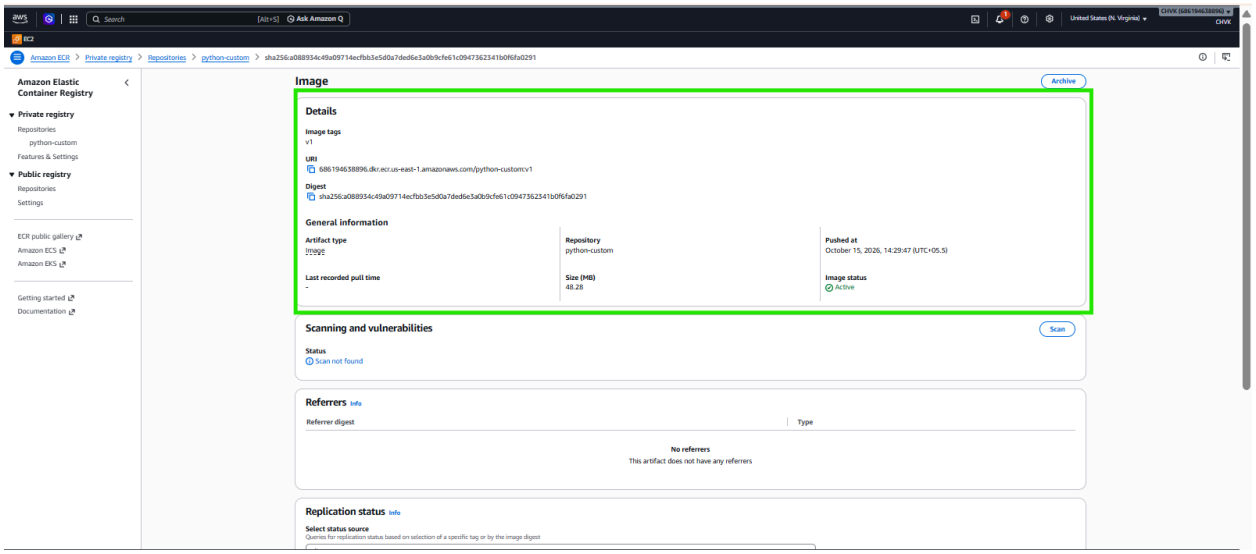
Sudo docker push 686194638896.dkr.ecr.us-east-1.amazonaws.com/python-custom:v1

Step 6: Verify

Open AWS Console → Amazon ECR → Repositories → "python-custom" → Images. Verify the "v1" tag.

Result

The same image has been pushed to Docker Hub and Amazon ECR.



TASK 4: INSTALL TERRAFORM AND AWS CLI ON THE LAPTOP, PROVISION AN EC2 INSTANCE, AND INSTALL JENKINS

Objective

To install Terraform and AWS CLI on the laptop, configure AWS access, create an EC2 instance using Terraform, and install Jenkins automatically.

PART A: INSTALL AWS CLI ON WINDOWS

Step 1: Download AWS CLI

1. Open your laptop.
2. Open Google Chrome.
3. Visit the official AWS CLI installation page: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
4. Find the Windows installation section.
5. Download the AWS CLI MSI installer for Windows (64-bit).

Step 2: Install AWS CLI

1. Open the downloaded ".msi" file.
2. Click Next.
3. Accept the license agreement.
4. Click Next.
5. Click Install.
6. Click Finish.

Step 3: Verify installation

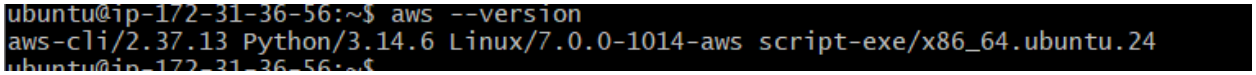
Close and reopen Command Prompt or PowerShell.

Run:

aws --version

Expected output begins with something similar to:

aws-cli/2.x.x



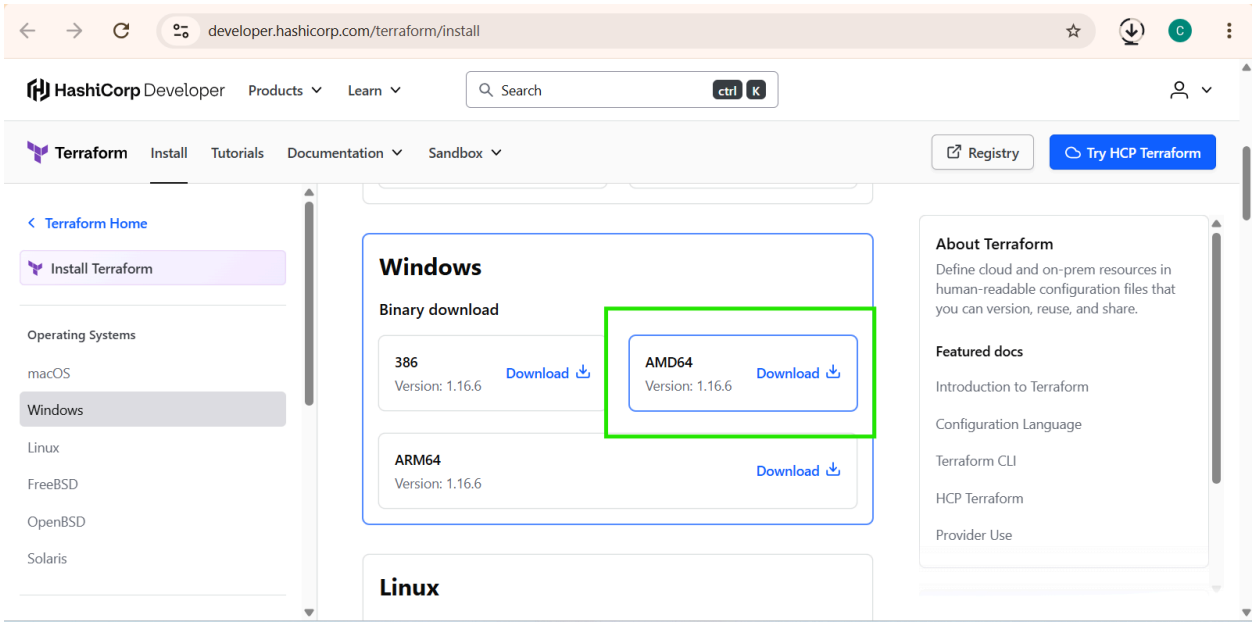
The exact version may differ.

If the command is not recognized, close the terminal, open it again, and verify the installation.

PART B: INSTALL TERRAFORM ON WINDOWS

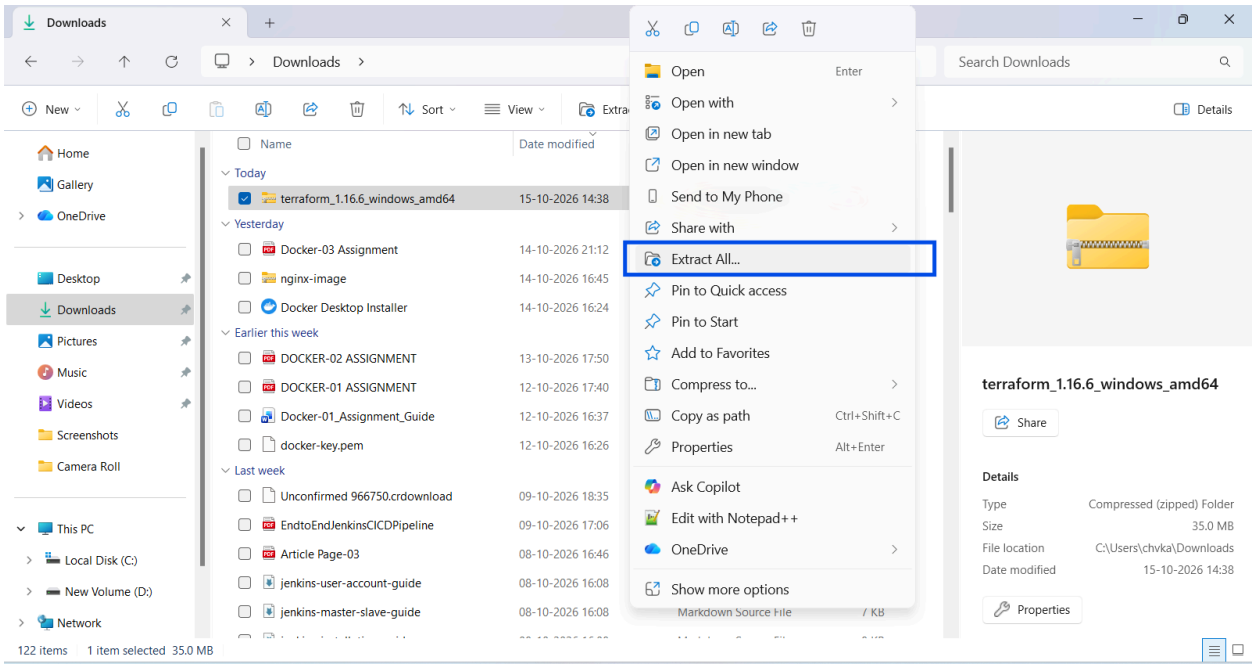
Step 1: Download Terraform

1. Open Google Chrome.
2. Visit: https://developer.hashicorp.com/terraform/install
3. Select Windows.
4. Download the appropriate 64-bit ZIP package.



Step 2: Extract Terraform

1. Open the downloaded ZIP file.
2. Extract "terraform.exe" into a folder such as:



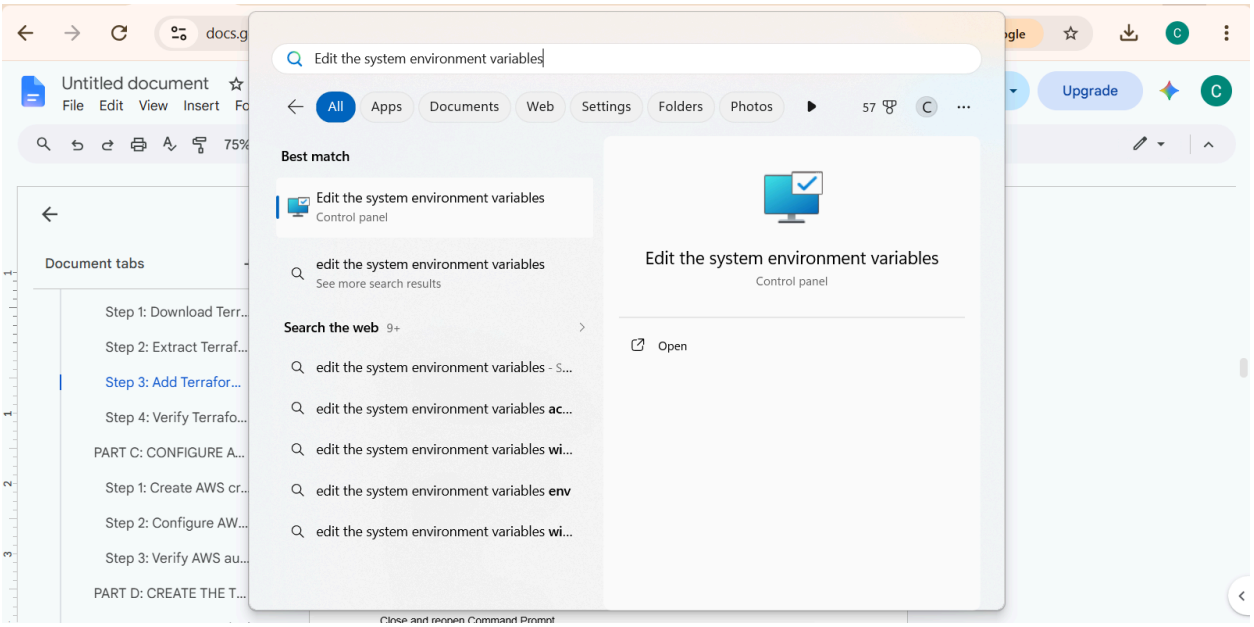
C:\Terraform

Ensure the file exists at:

C:\Terraform\terraform.exe

Step 3: Add Terraform to PATH

1. Click the Windows Start menu.
2. Search for Edit the system environment variables.



3. Open it.
4. Click Environment Variables.
5. Under User variables, select "Path".
6. Click Edit.
7. Click New.
8. Enter:

C:\Terraform

9. Click OK on all open windows.

Close and reopen Command Prompt.

Step 4: Verify Terraform

Execute:

.\terraform.exe -version

Expected output begins with:

Terraform v

```
PS C:\Terraform> .\terraform.exe -version
Terraform v1.16.6
on windows_amd64
```

The version number may differ.

PART C: CONFIGURE AWS CLI

Step 1: Create AWS credentials

1. Open AWS Console.
2. Sign in to your AWS account.
3. Open IAM.
4. Create or select an appropriate IAM identity for Terraform.
5. Grant only the permissions needed to create and manage the EC2 instance and security group for this task.
6. If using IAM user access keys, create an access key only if your organization's security policy allows it.

Prefer temporary credentials or an approved IAM Identity Center profile when available.

Never share your AWS access key or secret access key with anyone. Do not upload them to GitHub.

Step 2: Configure AWS CLI

Open Command Prompt or PowerShell.

Execute:

aws configure

Enter the requested values:

AWS Access Key ID: YOUR_ACCESS_KEY
AWS Secret Access Key: YOUR_SECRET_KEY
Default region name: us-east-1
Default output format: json

```
PS C:\Terraform> aws configure
AWS Access Key ID [****************QAWH]: AKIAZ7RDDIAYIHDTWH64
AWS Secret Access Key [****************IfRr]: ho08TSvQazGT1VribnUvRSV7yXKY3Txe005eUKrk
Default region name [us-east-1]: us-east-1
Default output format [json]: json
```

Use your own authorized credentials. Do not copy the example placeholders literally.

If your organization uses temporary credentials or IAM Identity Center, follow that authentication method instead of entering long-term access keys.

Step 3: Verify AWS authentication

aws sts get-caller-identity

If authentication is successful, AWS returns information about the current account and identity.

If you receive an error, check the credentials, permissions, and selected AWS profile before continuing.

```
PS C:\Terraform> aws sts get-caller-identity
{
    "UserId": "686194638896",
    "Account": "686194638896",
    "Arn": "arn:aws:iam::686194638896:root"
}

PS C:\Terraform>
```

PART D: CREATE THE TERRAFORM PROJECT

Step 1: Open a terminal

Open PowerShell or Command Prompt on your laptop.

Create a project folder:

mkdir terraform-jenkins
cd terraform-jenkins

```
PS C:\Terraform> mkdir terraform-jenkins


    Directory: C:\Terraform

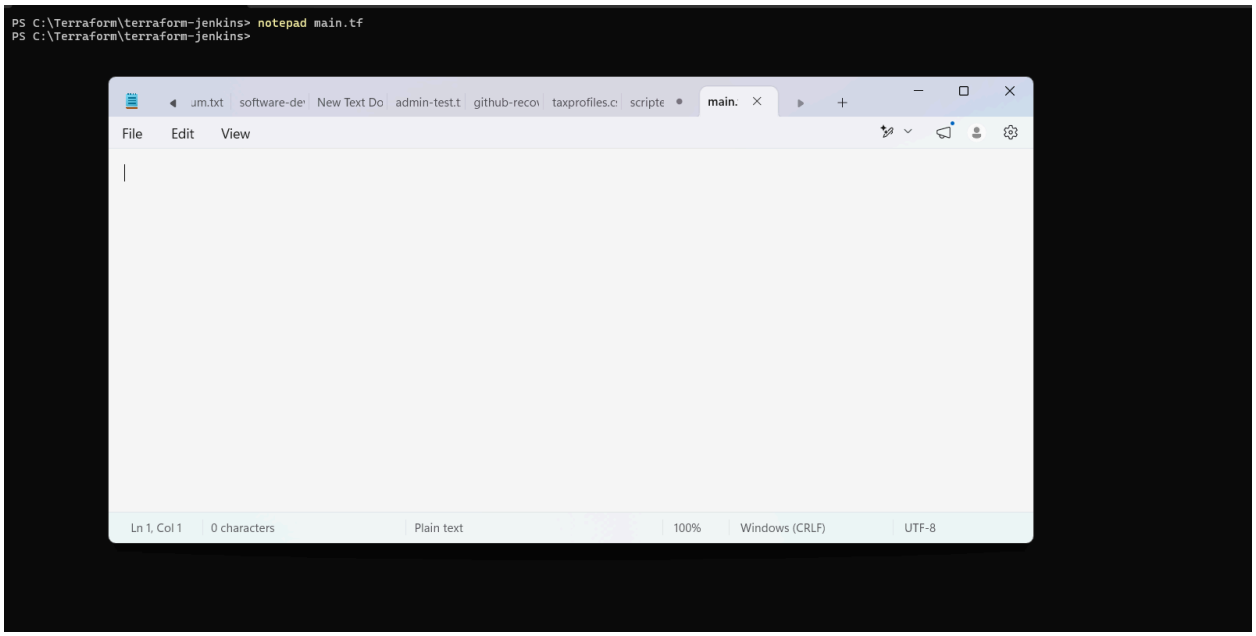

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        15-10-2026     15:04                terraform-jenkins


PS C:\Terraform> cd terraform-jenkins
PS C:\Terraform\terraform-jenkins>
```

Step 2: Create main.tf

Open Notepad:

notepad main.tf



Paste the following Terraform configuration:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
```

```
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

variable "my_ip" {
  type        = string
  description = "Your public IP address in CIDR format"
}

variable "key_name" {
  type        = string
  description = "Existing EC2 key pair name"
}

resource "aws_security_group" "jenkins_sg" {
  name   = "terraform-jenkins-sg"
  vpc_id = data.aws_vpc.default.id

  ingress {
    description = "SSH access"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    description = "Jenkins web access"
    from_port   = 8080
```

```
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "jenkins" {
  ami                         = data.aws_ami.ubuntu.id
  instance_type               = "t3.medium"
  subnet_id                   = data.aws_subnets.default.ids[0]
  vpc_security_group_ids      = [aws_security_group.jenkins_sg.id]
  key_name                    = var.key_name
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    set -eux

    apt-get update
    apt-get install -y fontconfig openjdk-21-jre curl

    install -m 0755 -d /etc/apt/keyrings

    curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key \
      -o /etc/apt/keyrings/jenkins-keyring.asc

    echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" \
      > /etc/apt/sources.list.d/jenkins.list

    apt-get update
    apt-get install -y jenkins
    systemctl enable --now jenkins
  EOF

  tags = {
    Name = "Terraform-Jenkins"
```

```
  }
}

output "jenkins_public_ip" {
  value = aws_instance.jenkins.public_ip
}
```

Save the file as "main.tf".

Verify the Jenkins repository signing-key URL against the official Jenkins Linux installation instructions before applying:

https://www.jenkins.io/doc/book/installing/linux/

Step 3: Create terraform.tfvars

Open Notepad:

notepad terraform.tfvars

Paste:

```
my_ip    = "YOUR_PUBLIC_IP/32"
key_name = "YOUR_EXISTING_KEY_PAIR_NAME"
```

Replace the placeholders:

- "YOUR_PUBLIC_IP/32": Your current public IP address in CIDR format.
- "YOUR_EXISTING_KEY_PAIR_NAME": The name of an existing EC2 key pair in "us-east-1".

Save the file.

Step 4: Verify project files

Execute:

dir

You should see:

main.tf

Terraform.tfvars

```
PS C:\Terraform\terraform-jenkins> dir


    Directory: C:\Terraform\terraform-jenkins


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        15-10-2026     15:17           2610 main.tf
-a----        15-10-2026     15:18             25 terraform.tfvars


PS C:\Terraform\terraform-jenkins>
```

PART E: EXECUTE TERRAFORM

Run all the following commands from the "terraform-jenkins" directory.

Step 1: Initialize Terraform

C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins init

```
PS C:\Terraform\terraform-jenkins> C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.69.0...
- Installed hashicorp/aws v6.69.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

This downloads the required AWS provider.

Step 2: Validate the configuration

C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins validate

```
PS C:\Terraform\terraform-jenkins> C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins validate
Success! The configuration is valid.
```

Expected result:

Success! The configuration is valid.

Step 3: Review the execution plan

C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins plan

```
PS C:\Terraform\terraform-jenkins> C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins plan
data.aws_vpc.default: Reading...
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 2s [id=ami-0fa5967347d08d2df]
data.aws_vpc.default: Read complete after 3s [id=vpc-041910120cd08c2c9]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 1s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.jenkins will be created
  + resource "aws_instance" "jenkins" {
      + ami                                  = "ami-0fa5967347d08d2df"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = true
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.medium"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "jenkins-key04"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
```

Review the resources Terraform intends to create.

Step 4: Create the EC2 instance

C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins apply

```
PS C:\Terraform\terraform-jenkins> C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins apply
data.aws_ami.ubuntu: Reading...
data.aws_vpc.default: Reading...
data.aws_ami.ubuntu: Read complete after 1s [id=ami-0fa5967347d08d2df]
data.aws_vpc.default: Read complete after 2s [id=vpc-041910120cd08c2c9]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 1s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.jenkins will be created
  + resource "aws_instance" "jenkins" {
      + ami                                  = "ami-0fa5967347d08d2df"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = true
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.medium"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "jenkins-key04"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
```

Read the plan carefully. Type "yes" only if you approve the resources and possible AWS charges.

Terraform creates the security group and EC2 instance.

Step 5: Get the public IP

Execute:

C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins output jenkins_public_ip

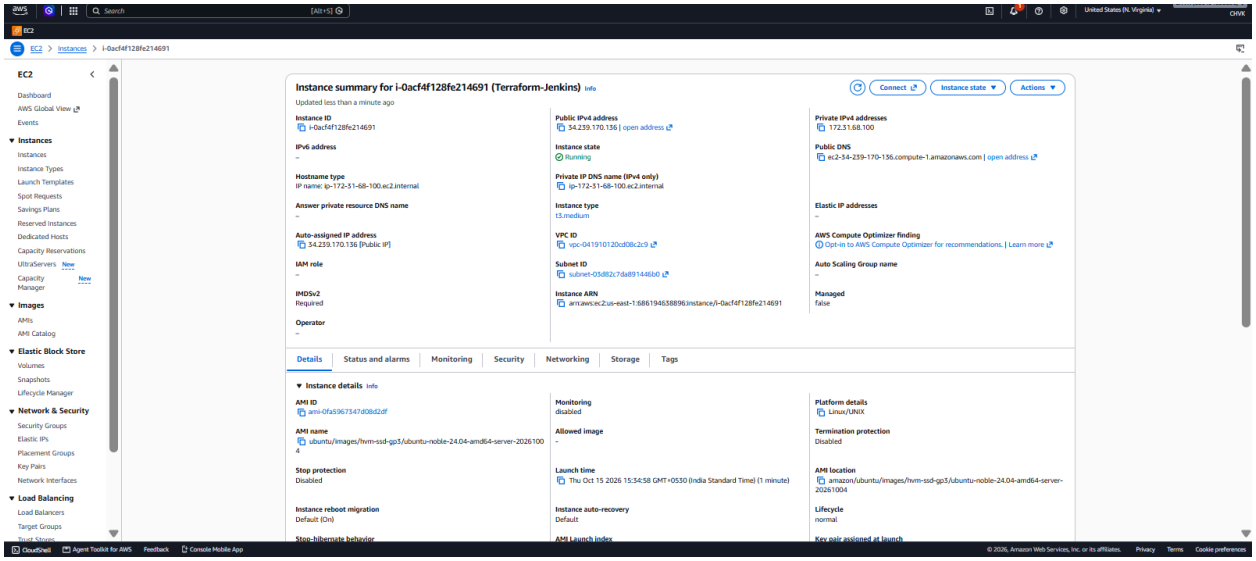
```
PS C:\Terraform\terraform-jenkins> C:\Terraform\terraform.exe -chdir=C:\Terraform\terraform-jenkins output jenkins_public_ip
"34.239.170.136"
PS C:\Terraform\terraform-jenkins>
```

Copy the "jenkins_public_ip" value.

PART F: VERIFY EC2 AND OPEN JENKINS

Step 1: Verify the instance

1. Open AWS Console.
2. Search for EC2.
3. Click Instances.
4. Find "Terraform-Jenkins".
5. Wait until the instance is Running.
6. Confirm that status checks pass.



Step 2: Open Jenkins

Open your browser and enter:

http://YOUR_TERRAFORM_EC2_PUBLIC_IP:8080

Replace the placeholder with the IP from Terraform.

The Jenkins installation may take several minutes after the instance first starts.

Step 3: Retrieve the initial administrator password

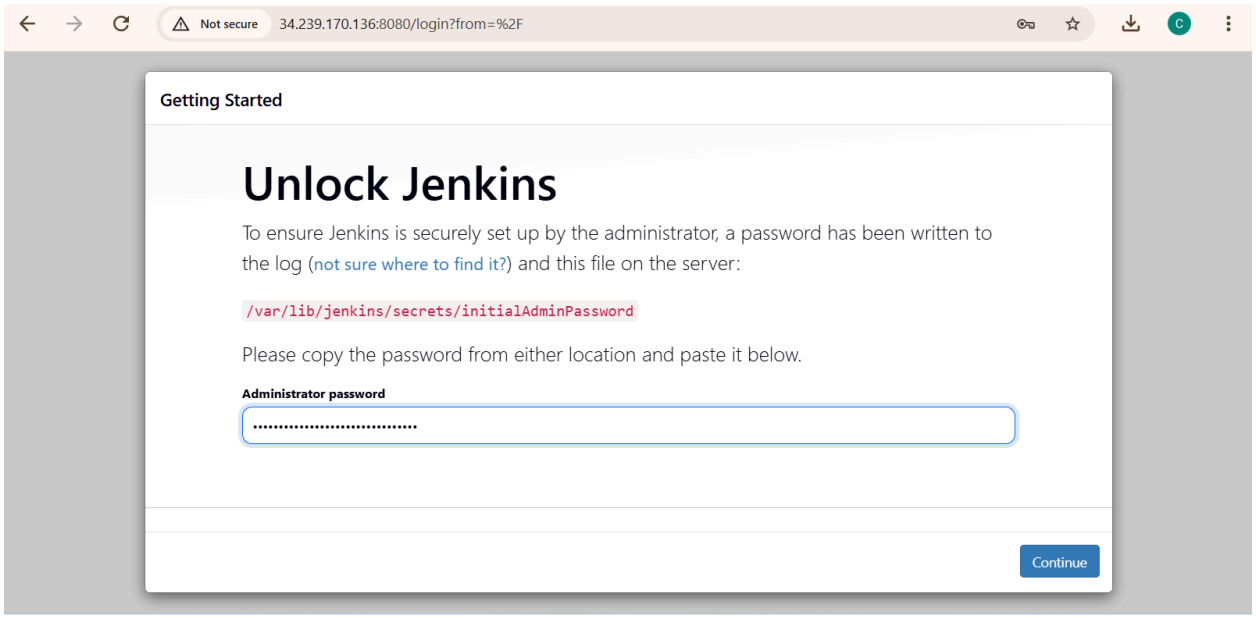
Connect to the new Ubuntu instance using SSH and the matching EC2 private key.

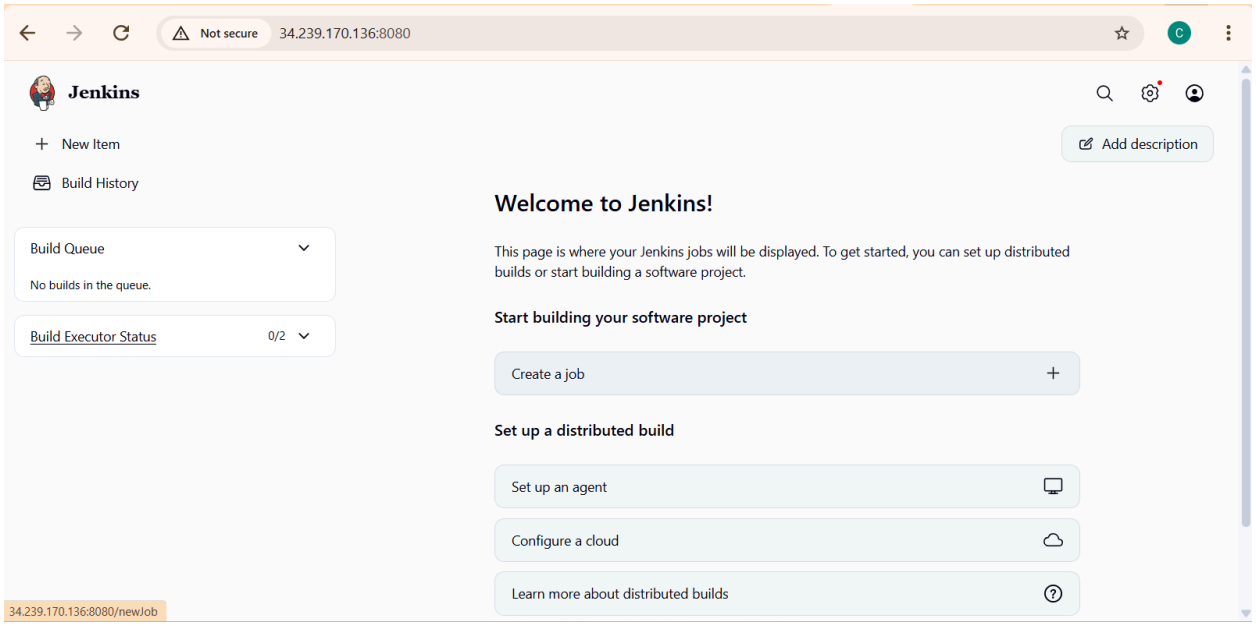
Run:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Copy the password and paste it into the Jenkins setup page.

Complete the setup wizard and create your Jenkins administrator account.





Result

Terraform and AWS CLI are installed on the laptop, AWS access is configured, a new EC2 instance has been provisioned, and Jenkins has been installed.

Important: This task creates a new EC2 instance and may incur charges. Do not delete the existing "Jenkins-04" or "Docker-server" instances.

TASK 5: CREATE A JENKINS JOB TO BUILD AND PUSH THE DOCKER IMAGE

Objective

To automate the Python Docker image build and push process using Jenkins Pipeline.

Step 1: Open Jenkins

Open the Jenkins URL of the instance you intend to use.

For the existing Jenkins instance:

http://44.222.224.225:8080

Alternatively, use the new Jenkins instance from Task 4.

Step 2: Prepare the Jenkins build environment

Ensure that the Jenkins build agent has Git and Docker installed and can access a Docker daemon.

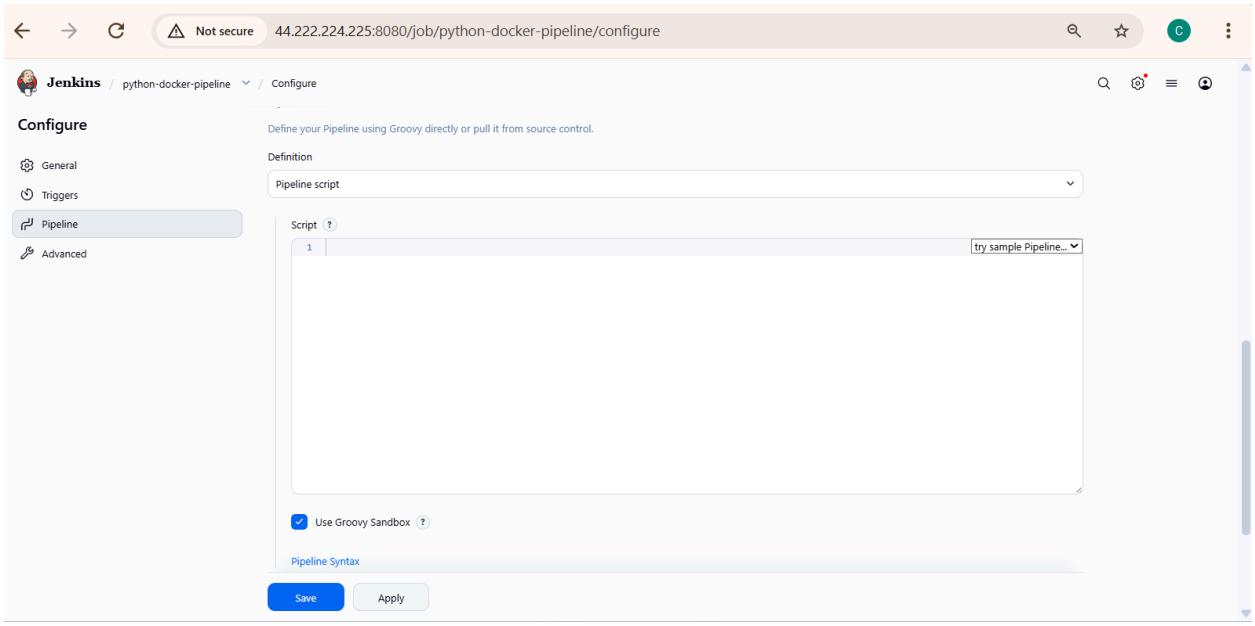
Install the Jenkins Pipeline and Credentials Binding plugins if they are missing.

Step 3: Add Docker Hub credentials

1. Open Jenkins Dashboard.
2. Click Manage Jenkins.
3. Click Credentials.
4. Open the appropriate global credentials store.
5. Click Add Credentials.
6. Select Username with password.
7. Enter username "chvkarthik".
8. Enter your Docker Hub access token as the password.
9. Set ID to "dockerhub-creds".
10. Click Create.

Step 4: Create the Pipeline job

1. Click New Item.
2. Enter "python-docker-pipeline".
3. Select Pipeline.
4. Click OK.
5. Scroll to the Pipeline section.
6. Select Pipeline script.



Step 5: Paste the Jenkinsfile

```
pipeline {
agent any

environment {
    IMAGE_NAME = 'chvkarthik/python-custom'
    IMAGE_TAG  = 'v1'
}

stages {
    stage('Checkout') {
        steps {
            git branch: 'main',
                url: 'https://github.com/betawins/Python-app.git'
        }
    }

    stage('Build Image') {
        steps {
            sh 'docker build -t IMAGE_NAME:{IMAGE_TAG} .'
        }
    }

    stage('Login to Docker Hub') {
        steps {
            withCredentials([
```

```
                usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DH_USER',
                    passwordVariable: 'DH_TOKEN'
                )
            ]) {
                sh '''
                    set +x
                    echo "$DH_TOKEN"|docker login -u "DH_USER" --password-stdin
                '''
            }
        }
    }

    stage('Push Image') {
        steps {
            sh 'docker push $IMAGE_NAME:{IMAGE_TAG}'
        }
    }

    stage('Logout') {
        steps {
            sh 'docker logout'
        }
    }
}

post {
    success {
        echo 'Image built and pushed successfully.'
    }
    failure {
        echo 'Pipeline failed. Check Console Output.'
    }
}

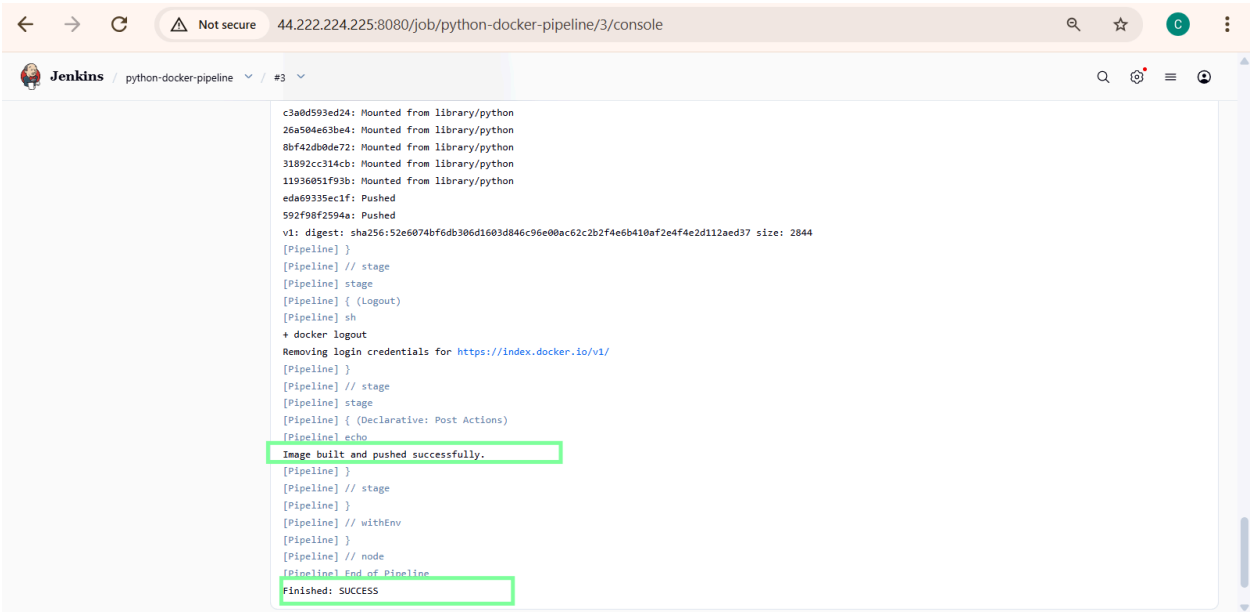
}
```

This example assumes the Dockerfile is in the root of the Python repository and Docker is available on the Jenkins build agent.

Step 6: Save and run

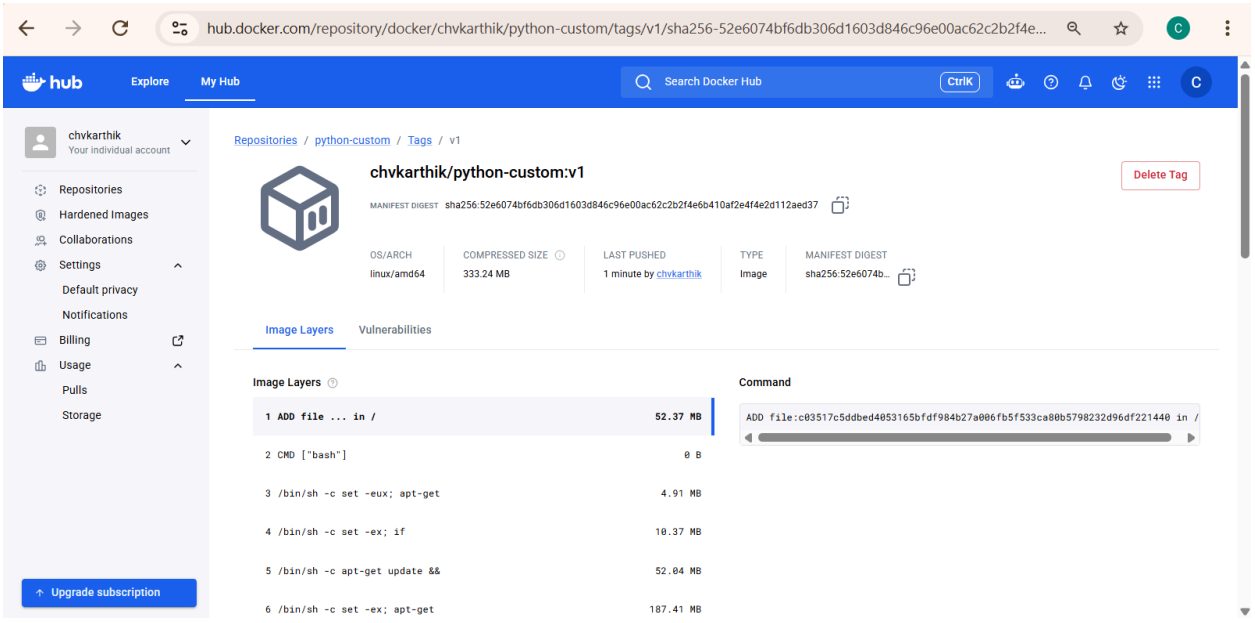
1. Click Save.

2. Click Build Now.
3. Open the build number.
4. Click Console Output.
5. Verify that all stages succeed.



Result

Jenkins builds the Python Docker image and pushes it to Docker Hub.



TASK 6: CREATE, BUILD, RUN, AND PUSH A FRONTEND DOCKER IMAGE

Objective

To containerize the frontend application and push it to Docker Hub.

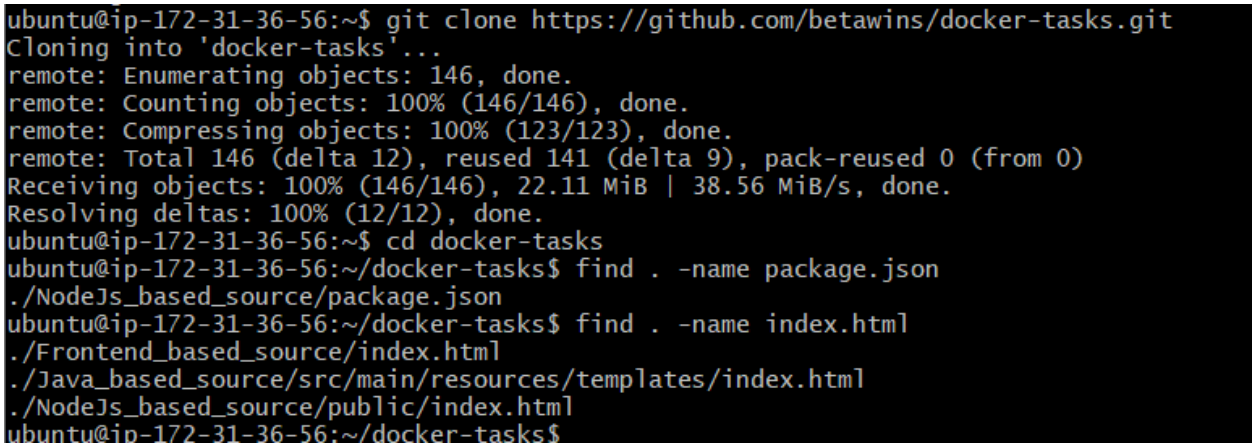
Step 1: Open the Docker-server terminal

AWS Console → EC2 → Instances → Docker-server → Connect.

Step 2: Clone the source repository

If you have already cloned the repository, reuse the existing folder. Otherwise:

```
git clone https://github.com/betawins/docker-tasks.git
cd docker-tasks
find . -name package.json
find . -name index.html
```



Change into the actual frontend application directory.

Step 3: Create Dockerfile

Go to

```
cd ~/docker-tasks/Frontend_based_source
```

Create a file

```
nano Dockerfile
```

Paste the code

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/
COPY style.css /usr/share/nginx/html/
COPY javascript.js /usr/share/nginx/html/
COPY todayDeal.js /usr/share/nginx/html/

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Save: `Ctrl + O` → `Enter` → `Ctrl + X`.

Step 4: Build and run

Sudo docker build -t frontend-app:v1 .
Sudo docker run -d --name frontend-app -p 8087:80 frontend-app:v1
Sudo docker ps
Sudo docker logs frontend-app
curl http://localhost:8087

```
ubuntu@ip-172-31-36-56:~/docker-tasks/Frontend_based_source$ curl http://localhost:8087
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="icon" href="https://upload.wikimedia.org/wikipedia/commons/4/4a/Amazon_icon.svg" />
    <title>Online Shopping site in India: Shop Online for Mobiles, Books, Watches, Shoes and More - Amazon.in</title>

    <!--css file-->
    <link rel="stylesheet" href="style.css" />

    <!--font awesome-->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.2.0/css/all.min.css"
        integrity="sha512-xh6O/CkQoPOWDdYTDqeRdPCVd1SpvCA9XXcUnZS2FmJNp1coAFzvtCN9BmamE+4aHK8yyUHUSCcJHgXloTyT2A=="
        crossorigin="anonymous" referrerpolicy="no-referrer" />

</head>

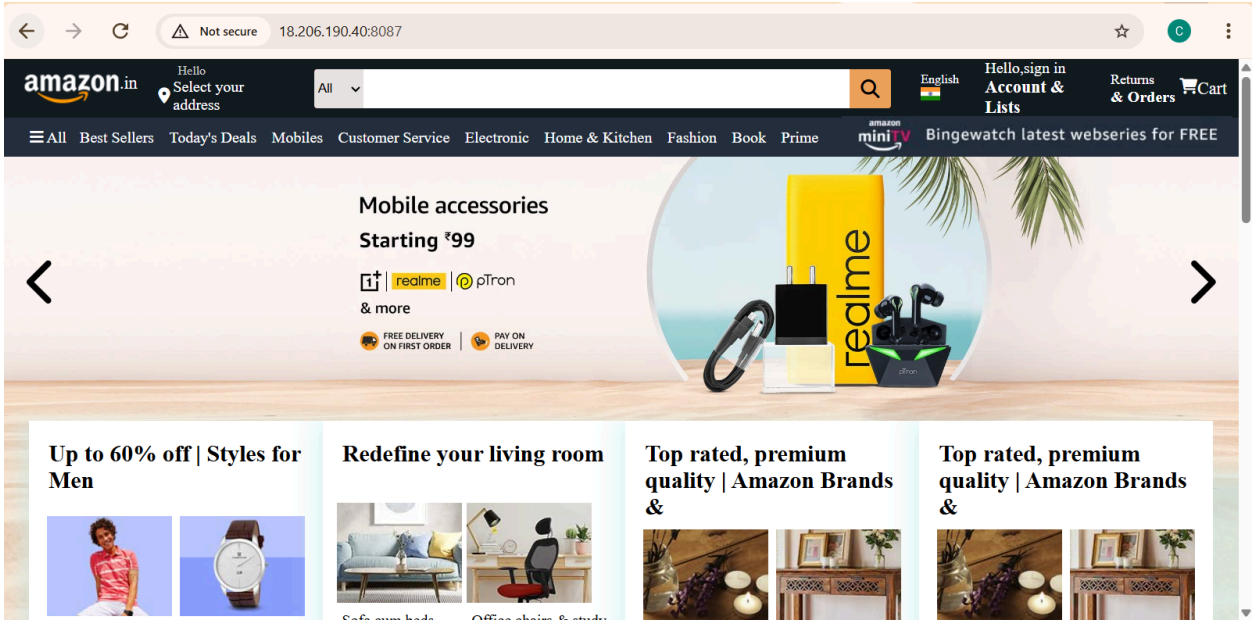
<body>
    <header class="header">
        <div class="container container-header">
            <div class="logo-container border-white">
                <div class="logo"></div>
                <span class="dotin">.in</span>
            </div>

            <div class="address-container border-white">
                <p class="hello">Hello</p>
                <div class="icon-address">
                    <i class="fa-solid fa-location-dot icon-location"></i>
                    <p>Select your address</p>
                </div>
            </div>

            <div class="search-container">
                <select class="search-select">
                    <option>All</option>
                </select>
                <input type="text" class="search-input" />
                <div class="search-icon">
                    <i class="fa-solid fa-magnifying-glass"></i>
                </div>

            </div>
```

http://PUBLIC_IP:8087



Step 5: Push to Docker Hub

Create a repository named "frontend-app" on Docker Hub.

Then execute:

Sudo docker login -u chvkarthik
Sudo docker tag frontend-app:v1 chvkarthik/frontend-app:v1
Sudo docker push chvkarthik/frontend-app:v1

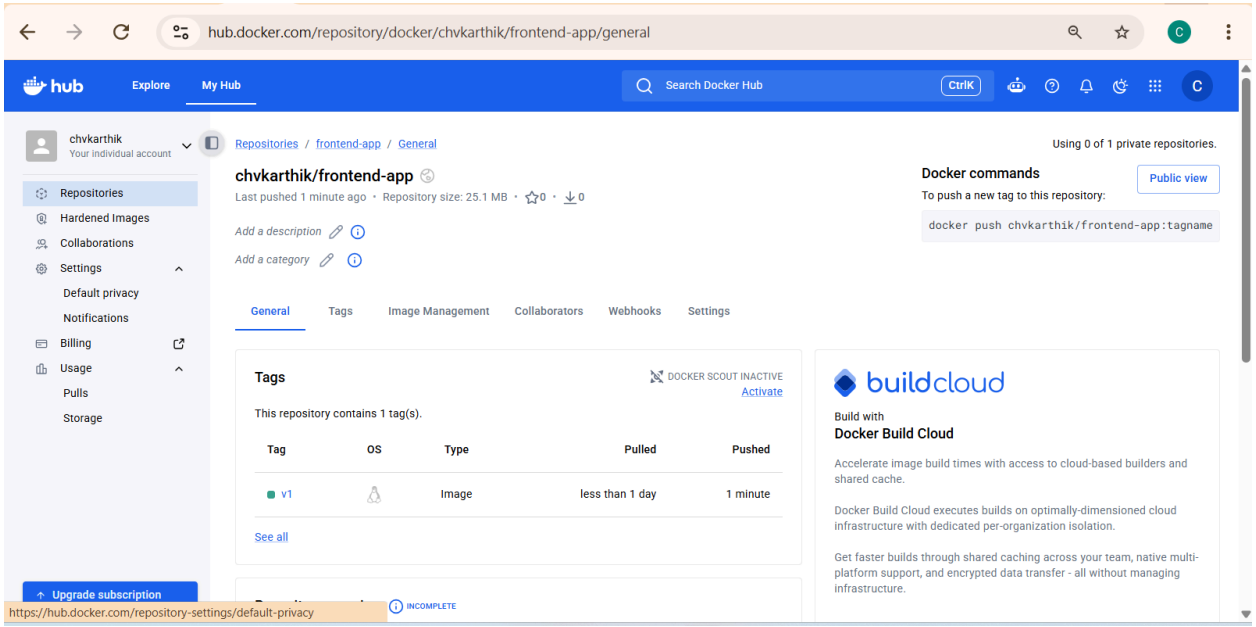
```
ubuntu@ip-172-31-36-56:~/docker-tasks/Frontend_based_source$ docker login -u chvkarthik

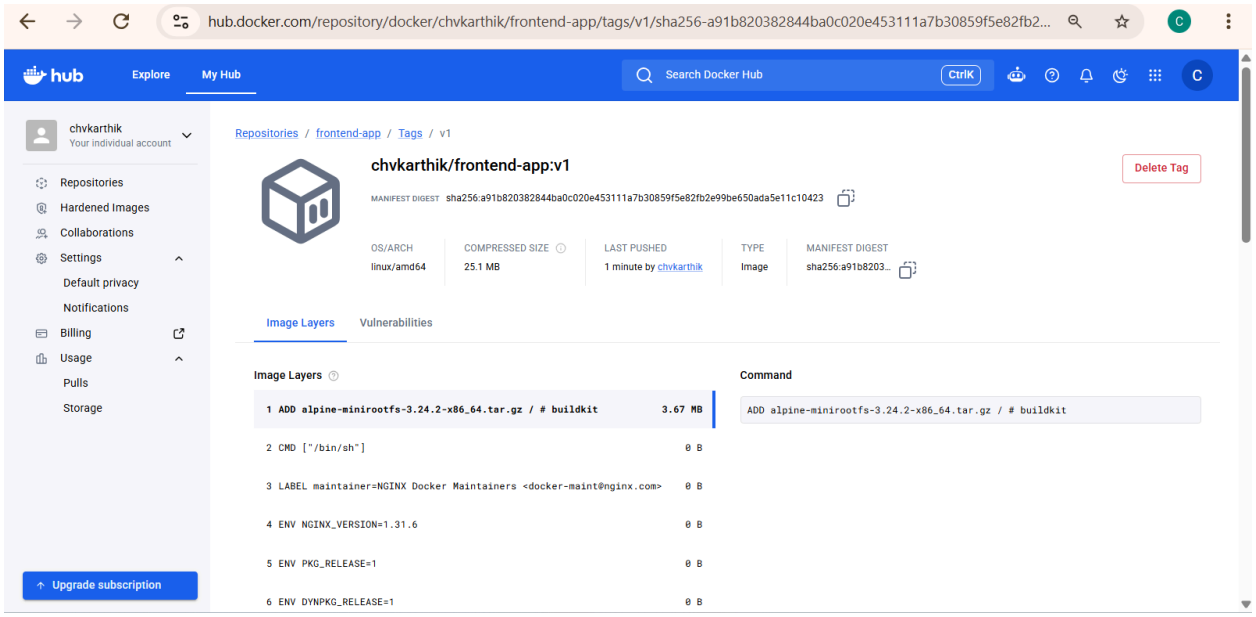
i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:
Login Succeeded
ubuntu@ip-172-31-36-56:~/docker-tasks/Frontend_based_source$ docker tag frontend-app:v1 chvkarthik/frontend-app:v1
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-36-56:~/docker-tasks/Frontend_based_source$ sudo docker tag frontend-app:v1 chvkarthik/frontend-app:v1
ubuntu@ip-172-31-36-56:~/docker-tasks/Frontend_based_source$ sudo docker push chvkarthik/frontend-app:v1
The push refers to repository [docker.io/chvkarthik/frontend-app]
64c8194480fe: Mounted from library/nginx
e72112c14215: Mounted from library/nginx
e2de96513ba9: Mounted from library/nginx
9dfbc7225b71: Pushed
d9aae54b5831: Mounted from library/nginx
9f0598db5759: Pushed
6c53d0b2a666: Mounted from library/nginx
745dfb2690dd: Mounted from library/nginx
128c09b5d5fc: Pushed
9a9a644fdd6a: Mounted from library/nginx
e76228b47809: Mounted from library/nginx
167c8866ec44: Pushed
v1: digest: sha256:a91b820382844ba0c020e453111a7b30859f5e82fb2e99be650ada5e11c10423 size: 2925
ubuntu@ip-172-31-36-56:~/docker-tasks/Frontend_based_source$
```

Result

The frontend image has been built, run, and pushed to Docker Hub.


hub.docker.com/repository/docker/chvkarthik/frontend-app/general
hub
Explore
My Hub
Search Docker Hub
CtrlK
chvkarthik
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / frontend-app / General
chvkarthik/frontend-app
Last pushed 1 minute ago • Repository size: 25.1 MB • ☆0 • ↓0
Add a description
Add a category
Using 0 of 1 private repositories.
Docker commands
To push a new tag to this repository:
Public view
docker push chvkarthik/frontend-app:tagname
General
Tags
Image Management
Collaborators
Webhooks
Settings
Tags
DOCKER SCOUT INACTIVE
Activate
This repository contains 1 tag(s).
Tag
OS
Type
Pulled
Pushed
v1
Image
less than 1 day
1 minute
See all
INCOMPLETE
buildcloud
Build with
Docker Build Cloud
Accelerate image build times with access to cloud-based builders and shared cache.
Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.
Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.
https://hub.docker.com/repository-settings/default-privacy


hub.docker.com/repository/docker/chvkarthik/frontend-app/tags/v1/sha256-a91b820382844ba0c020e453111a7b30859f5e82fb2...
hub
Explore
My Hub
Search Docker Hub
CtrlK
chvkarthik
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / frontend-app / Tags / v1
chvkarthik/frontend-app:v1
Delete Tag
MANIFEST DIGEST sha256:a91b820382844ba0c020e453111a7b30859f5e82fb2e99be650ada5e11c10423
OS/ARCH
linux/amd64
COMPRESSED SIZE
25.1 MB
LAST PUSHED
1 minute by chvkarthik
TYPE
Image
MANIFEST DIGEST
sha256:a91b8203...
Image Layers
Vulnerabilities
Image Layers
1 ADD alpine-minirootfs-3.24.2-x86_64.tar.gz / # buildkit 3.67 MB
2 CMD ["/bin/sh"] 0 B
3 LABEL maintainer=NGINX Docker Maintainers <docker-maint@nginx.com> 0 B
4 ENV NGINX_VERSION=1.31.6 0 B
5 ENV PKG_RELEASE=1 0 B
6 ENV DYNPKG_RELEASE=1 0 B
Command
ADD alpine-minirootfs-3.24.2-x86_64.tar.gz / # buildkit

TASK 7: CREATE, BUILD, RUN, AND PUSH A JAVA DOCKER IMAGE

Objective

To containerize the Java application and push it to Docker Hub.

Step 1: Find the Java project

You have already cloned the repository inside the `~/docker-tasks` folder.

First, navigate to the repository directory by running this command:

cd ~/docker-tasks

To find where the `pom.xml` file is located in the repository, run:

find . -name pom.xml

Based on the repository structure you previously found, the Java application is located inside the `Java_based_source` folder.

To verify the folder contents, run:

ls -la Java_based_source

If you can see the `pom.xml` file and the `src/` folder, you have located the Java project directory.

```
ubuntu@ip-172-31-36-56:~/docker-tasks/Frontend_based_source$ cd ~/docker-tasks
ubuntu@ip-172-31-36-56:~/docker-tasks$ find . -name pom.xml
./Java_based_source/pom.xml
ubuntu@ip-172-31-36-56:~/docker-tasks$ ls -la Java_based_source
total 16
drwxrwxr-x 3 ubuntu ubuntu 4096 Oct 15 10:20 .
drwxrwxr-x 6 ubuntu ubuntu 4096 Oct 15 10:20 ..
-rw-rw-r-- 1 ubuntu ubuntu 2366 Oct 15 10:20 pom.xml
drwxrwxr-x 3 ubuntu ubuntu 4096 Oct 15 10:20 src
```

Step 2: Create Dockerfile

Example for a Maven application producing a runnable JAR:

Create file

nano Dockerfile

Copy the following content

```
FROM maven:3.9-eclipse-temurin-21 AS builder

WORKDIR /app

COPY pom.xml .
COPY src ./src

RUN mvn -B package -DskipTests

FROM eclipse-temurin:21-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

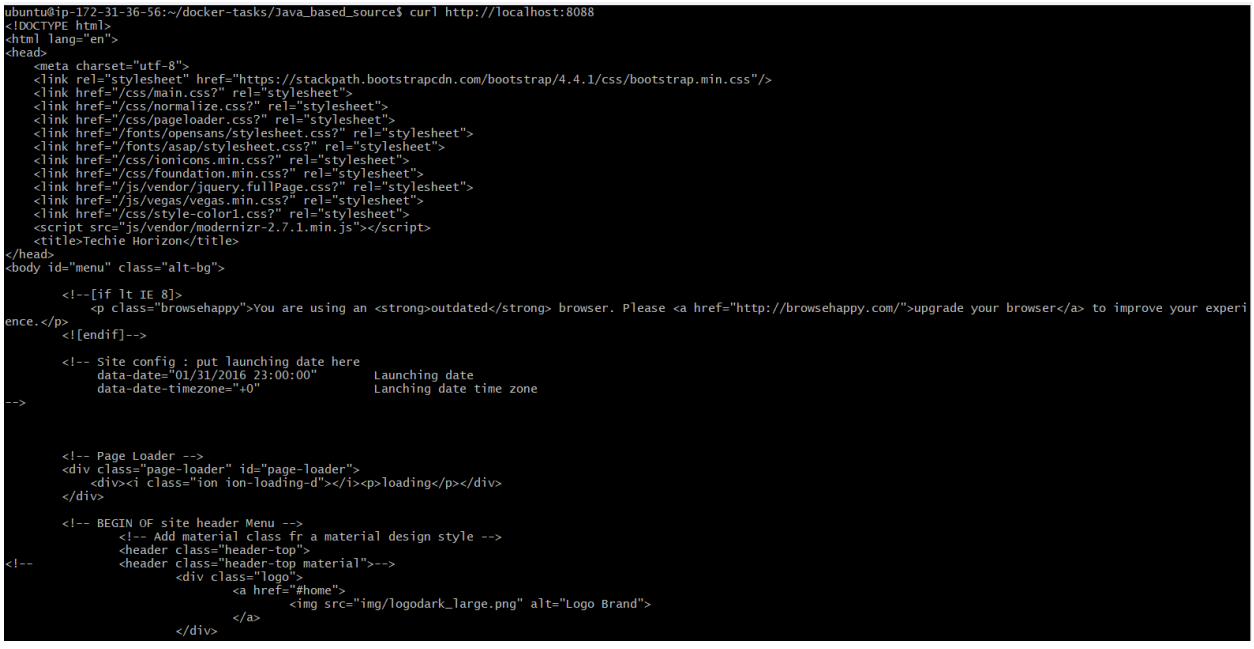
EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
```

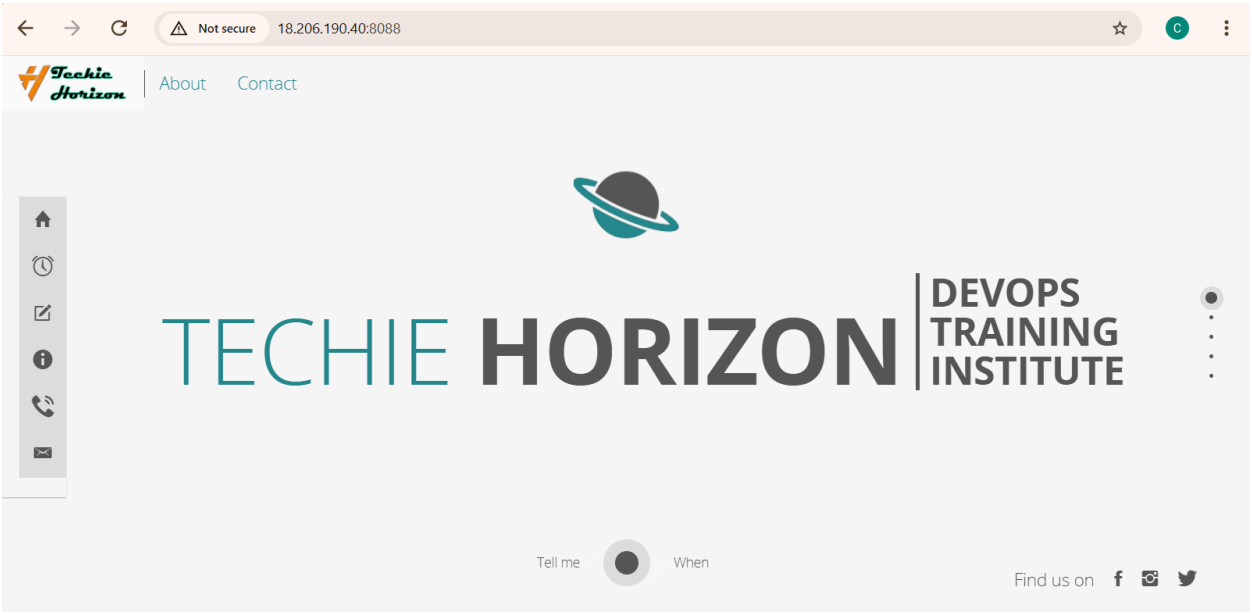
Save as "Dockerfile". Adjust the Java version, build command, and port if required.

Step 3: Build and run

```
Sudo docker build -t java-app:v1 .
Sudo docker run -d --name java-app -p 8088:8080 java-app:v1
Sudo docker ps
Sudo docker logs java-app
curl http://localhost:8083
```



http://PUBLIC_IP:8088



Step 4: Push to Docker Hub

Create a repository named "java-app".

Execute:

Sudo docker login -u chvkarthik
Sudo docker tag java-app:v1 chvkarthik/java-app:v1
Sudo docker push chvkarthik/java-app:v1

```
ubuntu@ip-172-31-36-56:~/docker-tasks/Java_based_source$ sudo docker login -u chvkarthik

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings

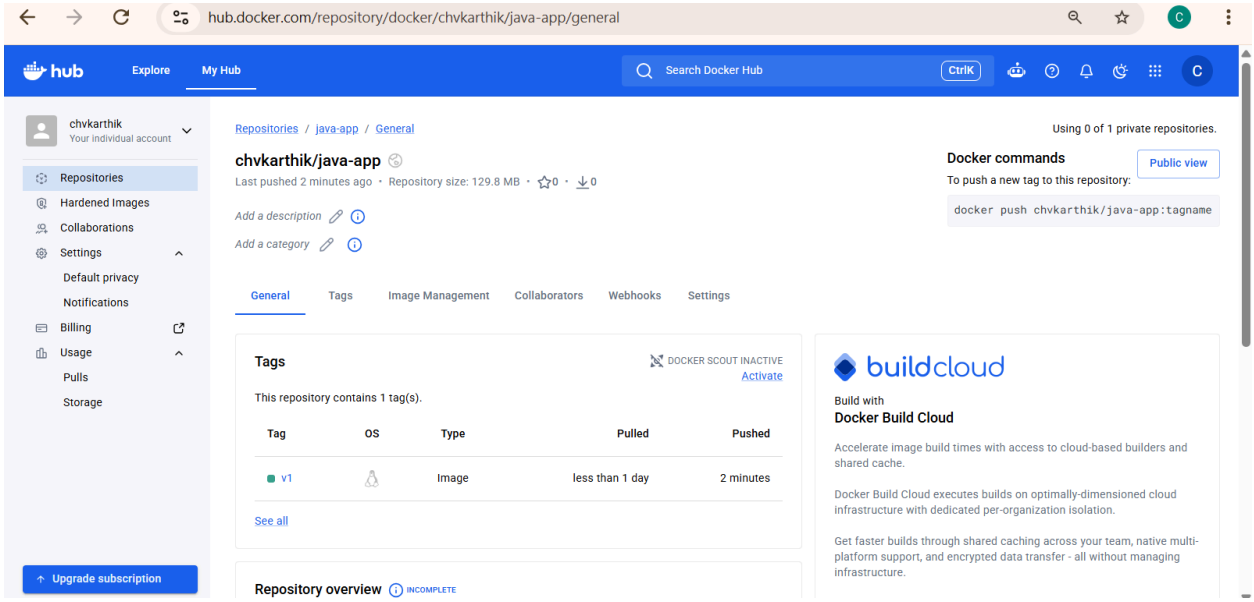

Password:

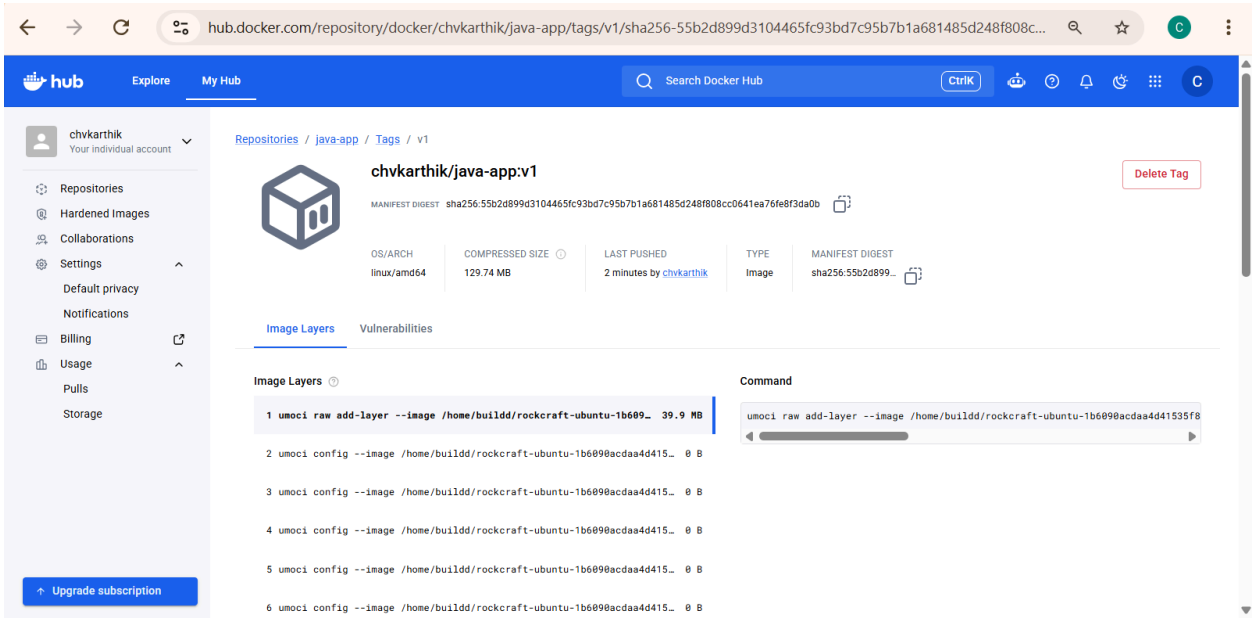
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-172-31-36-56:~/docker-tasks/Java_based_source$ sudo docker tag java-app:v1 chvkarthik/java-app:v1
ubuntu@ip-172-31-36-56:~/docker-tasks/Java_based_source$ sudo docker push chvkarthik/java-app:v1
The push refers to repository [docker.io/chvkarthik/java-app]
7fe32ee633a9: Pushed
09f4002ec090: Pushed
4e07a0f12b2c: Mounted from library/tomcat
06ad70e463aa: Mounted from library/tomcat
5d8dab7db274: Mounted from library/eclipse-temurin
9d8d42fbc560: Mounted from library/eclipse-temurin
8b4157e3a962: Mounted from library/eclipse-temurin
550bcd7b9380: Mounted from library/eclipse-temurin
v1: digest: sha256:55b2d899d3104465fc93bd7c95b7b1a681485d248f808cc0641ea76fe8f3da0b size: 2051
ubuntu@ip-172-31-36-56:~/docker-tasks/Java_based_source$
```

Result

The Java Docker image has been built, run, and pushed to Docker Hub.


hub.docker.com/repository/docker/chvkarthik/java-app/general
hub
Explore
My Hub
Search Docker Hub
CtrlK
chvkarthik
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / java-app / General
Using 0 of 1 private repositories.
chvkarthik/java-app
Last pushed 2 minutes ago • Repository size: 129.8 MB • ☆0 • ⭳0
Add a description
Add a category
Docker commands
Public view
To push a new tag to this repository:
docker push chvkarthik/java-app:tagname
General
Tags
Image Management
Collaborators
Webhooks
Settings
Tags
DOCKER SCOUT INACTIVE
Activate
This repository contains 1 tag(s).
Tag
OS
Type
Pulled
Pushed
v1
Image
less than 1 day
2 minutes
See all
buildcloud
Build with
Docker Build Cloud
Accelerate image build times with access to cloud-based builders and shared cache.
Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.
Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.
Repository overview
INCOMPLETE


hub.docker.com/repository/docker/chvkarthik/java-app/tags/v1/sha256-55b2d899d3104465fc93bd7c95b7b1a681485d248f808c...
hub
Explore
My Hub
Search Docker Hub
CtrlK
chvkarthik
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / java-app / Tags / v1
chvkarthik/java-app:v1
Delete Tag
MANIFEST DIGEST sha256:55b2d899d3104465fc93bd7c95b7b1a681485d248f808cc0641ea76fe8f3da0b
OS/ARCH
linux/amd64
COMPRESSED SIZE
129.74 MB
LAST PUSHED
2 minutes by chvkarthik
TYPE
Image
MANIFEST DIGEST
sha256:55b2d899...
Image Layers
Vulnerabilities
Image Layers
1 umoci raw add-layer --image /home/buildd/rockcraft-ubuntu-1b609… 39.9 MB
2 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d415… 0 B
3 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d415… 0 B
4 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d415… 0 B
5 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d415… 0 B
6 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d415… 0 B
Command
umoci raw add-layer --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d41535f8

TASK 8: CREATE, BUILD, RUN, AND PUSH A NODE.JS IMAGE WITH TAG v1

Objective

To build a Node.js Docker image with the "v1" tag, run it, and push it to Docker Hub.

Step 1: Find the Node.js source code

First, go to the `docker-tasks` directory on your Ubuntu EC2 instance.

cd ~/docker-tasks

List the files and folders:

ls

Locate the Node.js project directory. In your repository, it is named `NodeJs_based_source`.

Move into that directory:

cd NodeJs_based_source

Check the files:

ls

Look for the following files:

- `package.json` – Contains project information, dependencies, and scripts such as `npm start`.
- `package-lock.json` – Records the exact dependency versions installed for the project, helping ensure consistent installations.

Check the `package.json` file:

cat package.json

Look for the `scripts` section to identify the application start command. For example:

"scripts": {

  "start": "node src/index.js"

}

```
ubuntu@ip-172-31-36-56:~/docker-tasks/Java_based_source$ cd ~/docker-tasks
ubuntu@ip-172-31-36-56:~/docker-tasks$ ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
ubuntu@ip-172-31-36-56:~/docker-tasks$ cd NodeJs_based_source
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ ls
package-lock.json  package.json  public  src
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ cat package.json
{
  "name": "zomato_clone",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@emotion/react": "^11.10.5",
    "@emotion/styled": "^11.10.5",
    "@mui/icons-material": "^5.11.0",
    "@mui/material": "^5.11.1",
    "@testing-library/jest-dom": "^5.16.5",
    "@testing-library/react": "^13.4.0",
    "@testing-library/user-event": "^13.5.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "sass": "^1.57.1",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "eslintConfig": {
    "extends": [
      "react-app",
      "react-app/jest"
    ]
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all"
    ],
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
```

Step 2: Create Dockerfile

Inside the `NodeJs_based_source` directory, create a file named `Dockerfile`:

nano Dockerfile

Add the following content:

```
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Save the file. If you are using Nano, press `Ctrl + O`, then `Enter`, and press `Ctrl + X` to exit.

Explanation of each Dockerfile instruction

Instruction	Purpose
`FROM node:22-alpine`	Uses Node.js 22 with Alpine Linux as the base image.
`WORKDIR /app`	Sets `/app` as the working directory inside the container.
`COPY package*.json ./`	Copies `package.json` and, if present, `package-lock.json` into the container.
`RUN npm ci`	Installs dependencies using the lock file. This requires a valid `package-lock.json`.
`COPY . .`	Copies the remaining application files into the container.
`EXPOSE 3000`	Documents that the application is expected to use port `3000` inside the container.
`CMD ["npm", "start"]`	Runs the application's start script when the container starts.

Step 3: Build with tag v1

Sudo docker build -t node-app:v1 .
Sudo docker images

```
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ sudo docker images
                                                                                                                    i Info →   U  In Us
IMAGE                                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
686194638896.dkr.ecr.us-east-1.amazonaws.com/myapp:latest      1586db5eaf15       503MB         135MB    U
686194638896.dkr.ecr.us-east-1.amazonaws.com/python-custom:v1  a088934c49a0       198MB        48.3MB    U
alpine-tomcat:v1                                               9b13fdf855d3       317MB        86.6MB    U
chvkarthik/frontend-app:v1                                     a91b82038284      93.7MB        26.3MB    U
chvkarthik/java-app:v1                                         55b2d899d310       504MB         136MB    U
chvkarthik/python-custom:v1                                    a088934c49a0       198MB        48.3MB    U
eclipse-temurin:21-jre                                         ccbad9e7cb84       472MB         123MB
frontend-app:v1                                                a91b82038284      93.7MB        26.3MB    U
java-app:v1                                                    55b2d899d310       504MB         136MB    U
maven:3.9-eclipse-temurin-21                                   99e61abcff91       814MB         250MB
multi-stage-example:multi                                      1586db5eaf15       503MB         135MB    U
multi-stage-example:single                                     69efd96df1d5       988MB         324MB    U
mysql:8.0                                                      7dcddc01f13b       1.1GB         249MB    U
nginx:alpine                                                   df2210b830e1      94.4MB        27.2MB    U
node-app:v1                                                    71567cbadaee      1.07GB         190MB
node:22-alpine                                                 0a7108bf6c7b       238MB        61.1MB
python-custom:v1                                               a088934c49a0       198MB        48.3MB    U
python:3.12-slim                                               a6e34c598f24       179MB        45.4MB
redis:alpine                                                   3811787313eb       161MB        39.6MB    U
ubuntu:latest                                                  f144425ff09b       162MB        45.6MB    U
wordpress:latest                                               81a34254acaa      1.12GB         284MB    U
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$
```

Step 4: Run the container

Sudo docker run -d --name node-app -p 8090:3000 node-app:v1
Sudo docker ps
Sudo docker logs node-app

```
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ sudo docker run -d --name node-app -p 8090:3000 node-app:v1
4923f428474e551ae3a68e40c5bd926bdb4e1c0a5e2675c3a23a16a23d579333
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ sudo docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED          STATUS          PORTS                                                   NAMES
4923f428474e   node-app:v1       "docker-entrypoint.s…"   17 seconds ago   Up 16 seconds   0.0.0.0:8090->3000/tcp, [::]:8090->3000/tcp             node-app
d34f43972c08   java-app:v1       "/__cacert_entrypoin…"   23 minutes ago   Up 23 minutes   0.0.0.0:8088->8080/tcp, [::]:8088->8080/tcp             java-app
c127746a8126   frontend-app:v1   "/docker-entrypoint.…"   40 minutes ago   Up 40 minutes   0.0.0.0:8087->80/tcp, [::]:8087->80/tcp                 frontend-app
b93e93e09336   python-custom:v1  "python app.py"          3 hours ago      Up 3 hours      5000/tcp, 0.0.0.0:5000->8080/tcp, [::]:5000->8080/tcp   python-app
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ sudo docker logs node-app

> zomato_clone@0.1.0 start
> react-scripts start

Browserslist: caniuse-lite is outdated. Please run:
  npx update-browserslist-db@latest
  Why you should do it regularly: https://github.com/browserslist/update-db#readme
(node:25) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
(Use `node --trace-deprecation ...` to show where the warning was created)
(node:25) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...

Compiled successfully!

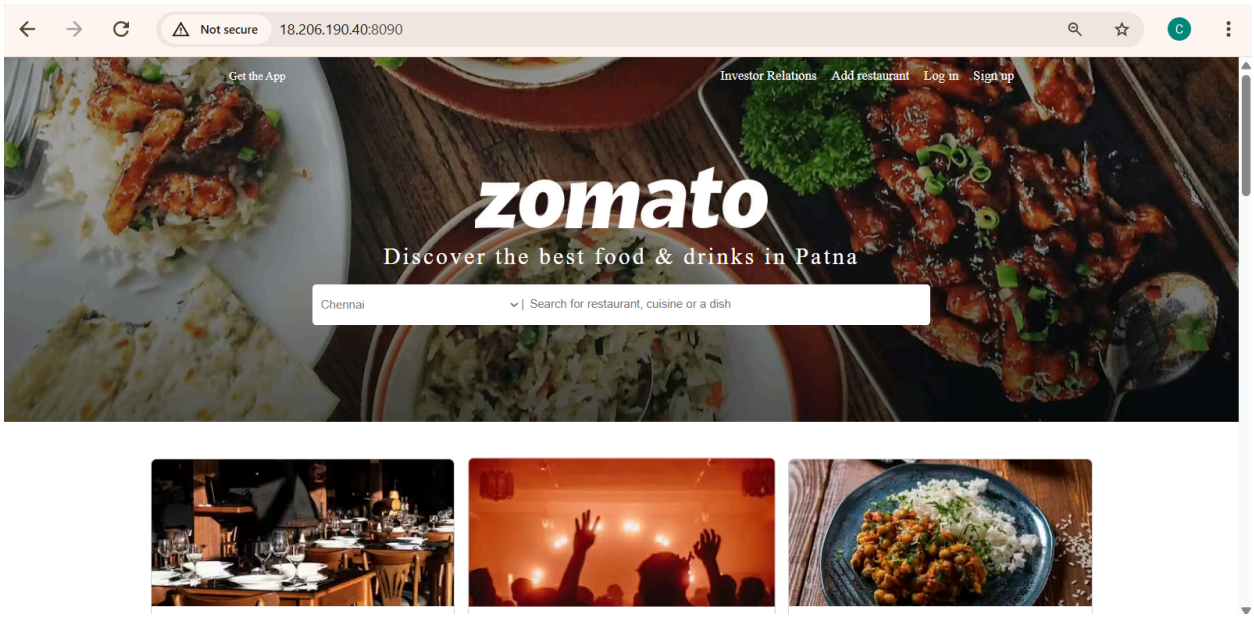
You can now view zomato_clone in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://172.17.0.5:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
Compiling...
Compiled successfully!
webpack compiled successfully
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$
```

http://PUBLIC_IP:8090



Step 5: Push to Docker Hub

Create a repository named "node-app".

Execute:

Sudo docker login -u chvkarthik
Sudo docker tag node-app:v1 chvkarthik/node-app:v1
Sudo docker push chvkarthik/node-app:v1

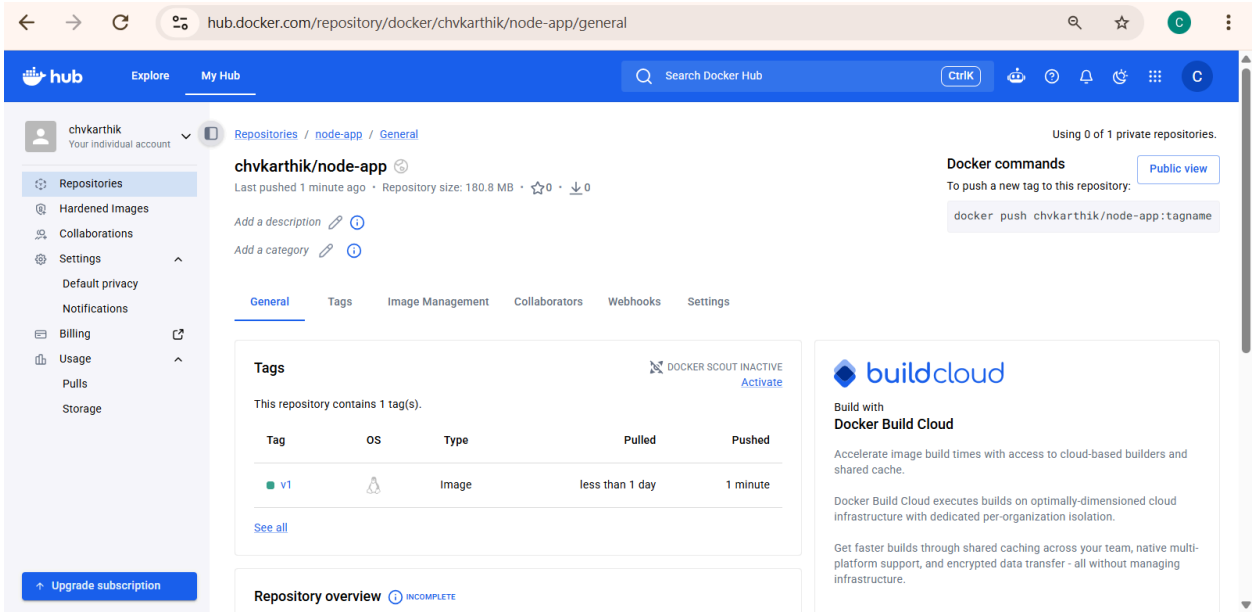
```
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ sudo docker login -u chvkarthik

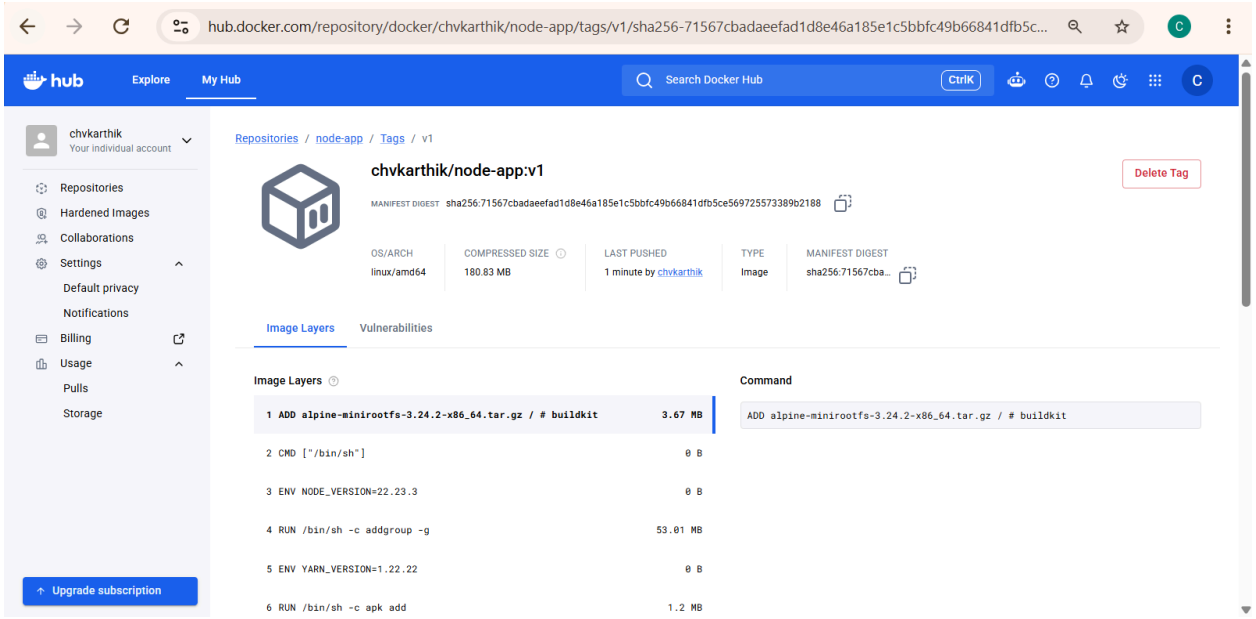
i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:
Login Succeeded
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ docker tag node-app:v1 chvkarthik/node-app:v1
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ sudo docker tag node-app:v1 chvkarthik/node-app:v1
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$ sudo docker push chvkarthik/node-app:v1
The push refers to repository [docker.io/chvkarthik/node-app]
87dc9a1502e6: Pushed
bb3656ca9f21: Pushed
e554276b05e6: Mounted from library/node
d39db1cf9caa: Mounted from library/node
5bb039cd1ad4: Pushed
e2de96513ba9: Mounted from chvkarthik/frontend-app
5252fa156749: Pushed
f7f2d304681a: Mounted from library/node
v1: digest: sha256:71567cbadaeefad1d8e46a185e1c5bbfc49b66841dfb5ce569725573389b2188 size: 2062
ubuntu@ip-172-31-36-56:~/docker-tasks/NodeJs_based_source$
```

Result

The Node.js Docker image has been built with the "v1" tag, run, and pushed to Docker Hub.





TASK 9: SET UP WORDPRESS AND MYSQL USING DOCKER COMPOSE

Objective

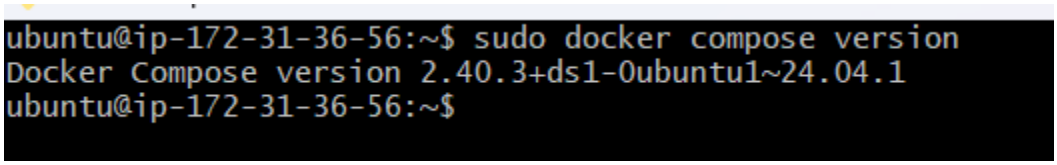
To deploy WordPress and MySQL together using Docker Compose.

Step 1: Connect to Docker-server

AWS Console → EC2 → Instances → Docker-server → Connect.

Verify Docker Compose:

Sudo docker compose version



Step 2: Create a directory

mkdir wordpress-compose
cd wordpress-compose

Step 3: Create the Compose file

nano compose.yaml

Paste:

```
services:
  wordpress:
    image: wordpress:latest
    container_name: wordpress
    restart: unless-stopped
    ports:
      - "8091:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_NAME: wordpress
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: "WpUser!7xQ9#vL2mR8"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - wordpress_data:/var/www/html

  db:
    image: mysql:8.4
```

```
    container_name: wordpress-db
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: "WpUser!7xQ9#vL2mR8"
      MYSQL_ROOT_PASSWORD: "MySQLRoot!4pN8@zK6wT"
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s

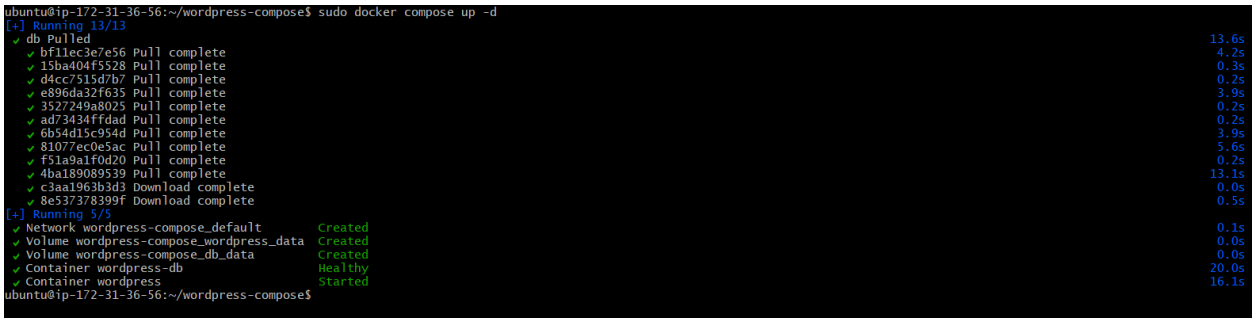
volumes:
  wordpress_data:
  db_data:
```

Replace both example passwords with strong passwords. The WordPress and MySQL user passwords must match.

Save using Ctrl + O, Enter, and Ctrl + X.

Step 4: Start WordPress and MySQL

Sudo docker compose up -d



Step 5: Verify the containers

Sudo docker compose ps
Sudo docker compose logs
Sudo docker compose logs wordpress
Sudo docker compose logs db

Step 6: Open WordPress

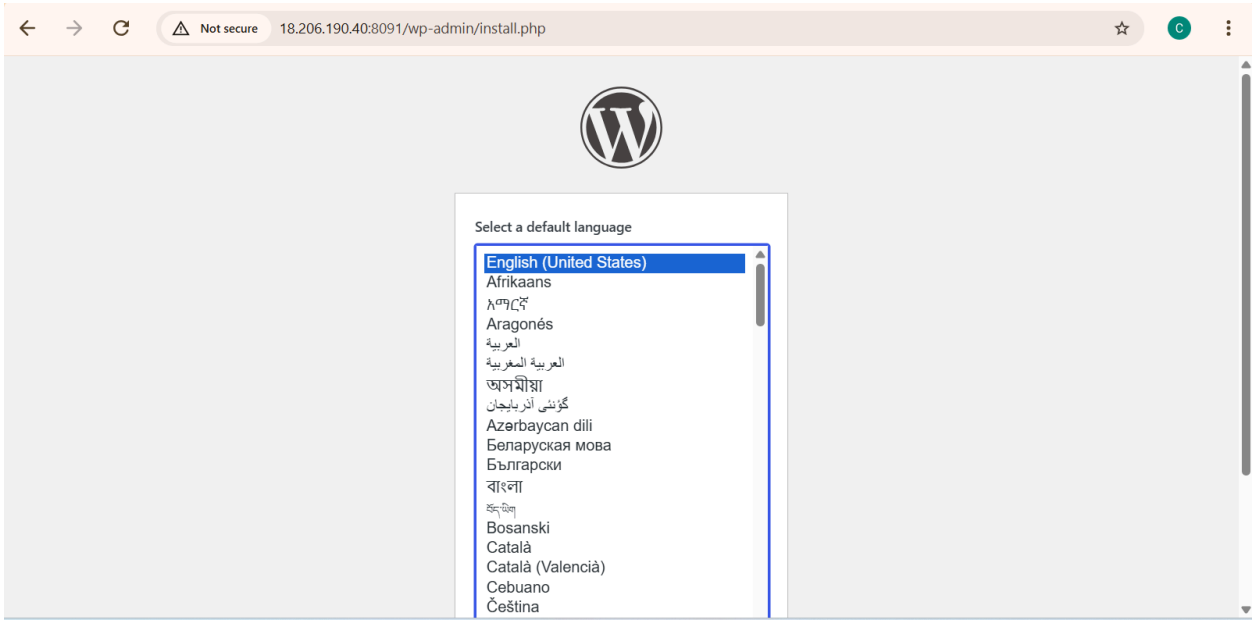
In your browser, enter:

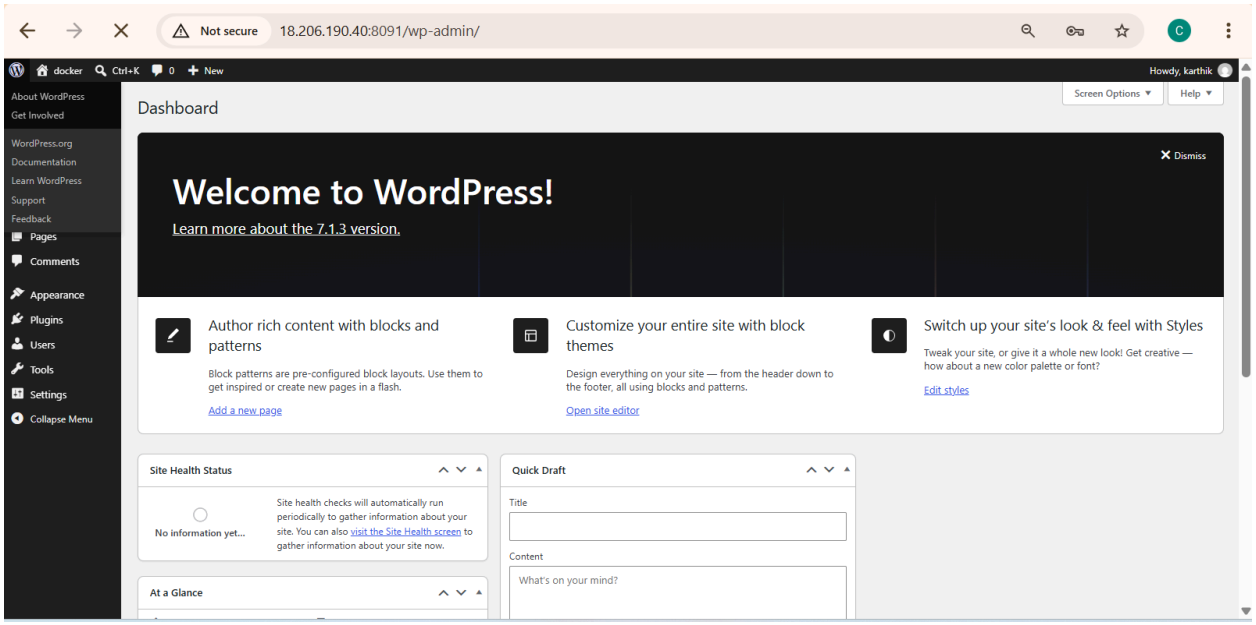
http://YOUR_DOCKER_EC2_PUBLIC_IP:8085

Replace the placeholder with your Docker-server public IP.

If remote access is needed, add inbound TCP port 8085 to the Docker-server security group and restrict the source to your public IP. Do not expose MySQL port 3306 publicly.

Complete the WordPress setup wizard.





Step 7: Stop and restart the services

Stop:

Sudo docker compose stop

```
ubuntu@ip-172-31-36-56:~/wordpress-compose$ sudo docker compose stop
[+] Stopping 2/2
 ✔ Container wordpress     Stopped
 ✔ Container wordpress-db  Stopped
ubuntu@ip-172-31-36-56:~/wordpress-compose$
```

Start:

Sudo docker compose start

```
ubuntu@ip-172-31-36-56:~/wordpress-compose$ sudo docker compose start
[+] Running 2/2
 ✔ Container wordpress-db  Healthy
 ✔ Container wordpress     Started
ubuntu@ip-172-31-36-56:~/wordpress-compose$
```

Remove containers and the network:

Sudo docker compose down

```
ubuntu@ip-172-31-36-56:~/wordpress-compose$ sudo docker compose down
[+] Running 3/3
 ✔ Container wordpress                 Removed
 ✔ Container wordpress-db              Removed
 ✔ Network wordpress-compose_default   Removed
ubuntu@ip-172-31-36-56:~/wordpress-compose$
```

Named volumes retain the data. Do not use `docker compose down -v` unless you intentionally want to delete the stored WordPress and database data.

Result

WordPress and MySQL have been deployed together using Docker Compose.

FINAL VERIFICATION CHECKLIST

Docker

[ ] Python Docker image created.

[ ] Python image pushed to Docker Hub.

[ ] Same Python image pushed to Amazon ECR.

[ ] Frontend image built, run, and pushed.

[ ] Java image built, run, and pushed.

[ ] Node.js image built with tag v1, run, and pushed.

Terraform and Jenkins

[ ] Terraform installed on the laptop.

[ ] AWS CLI installed on the laptop.

[ ] AWS credentials configured securely.

[ ] Terraform configuration initialized and validated.

[ ] New EC2 instance created through Terraform.

[ ] Jenkins installed and accessible.

[ ] Jenkins Pipeline job created and executed successfully.

Docker Compose

[ ] WordPress container running.

[ ] MySQL container healthy.

[ ] WordPress setup page accessible.

[ ] Named volumes created for persistent data.

Conclusion

This challenge demonstrates Docker image creation, Docker Hub and Amazon ECR integration, Terraform-based EC2 provisioning, Jenkins automation, application containerization, and WordPress deployment using Docker Compose.